

Towards five-state minimal-time firing squads: two barriers on the half-line and certified bounds

Kia Keng Giam^{a,*}, Beichen Sun^b, Yaoqi Zhao^b

^a*Temasek Junior College, Singapore*

^b*Hwa Chong Institution, Singapore*

Abstract

Whether the firing squad synchronization problem has a minimal-time solution with five states has been open since Mazoyer’s six-state solution of 1987. We develop tools for minimal-time solutions with any number of states and measure how far they reach on this question. All lines are described by one half-line diagram and one reflected triangle per length, and every triangle is a function of two anti-diagonals of the half-line. On the half-line we prove two necessary conditions: a pumping lemma, which forces its non-periodic part to reach the line $i = t/3$, and a left-border lemma, which forbids periodic behaviour in any wedge at the border; in particular the half-line of a minimal-time solution is never eventually regular. A propositional encoding, validated against Mazoyer’s solution, gives an LRAT-certified proof, checked in six seconds, that no four-state minimal-time solution exists. For five states we exhibit rules that synchronize every line of length at most 14 in minimal time, and we show with certified solver calls that every five-state minimal-time solution uses at least 15 distinct neighbourhoods on its half-line below the anti-diagonal $t + i = 78$. The half-lines of low complexity that survive all our tests are irregular, and search-size estimates of about 25 nodes for four states against 10^{11} to 10^{19} for five quantify why the question remains open.

Keywords: firing squad synchronization problem, cellular automata, state complexity, minimal time, SAT solving, proof certificates

*Corresponding author.

Email address: giamkiakeng@gmail.com (Kia Keng Giam)

Notation at a glance

The table lists the notation that the paper uses throughout; every term and symbol, including those that occur only in one proof, is defined in the glossary of Appendix A. Sans-serif capitals always denote states.

notation	meaning
$S, k = S $	set of states and number of states
$L, G, F; A, B$	quiescent state, general, firing state; the two auxiliary states when $k = 5$
$W = S \setminus \{F\}, *$	working states; the border
δ	a rule: $(W \cup \{*\}) \times W \times (W \cup \{*\}) \rightarrow S$ with $\delta(L, L, L) = \delta(L, L, *) = L$
$C_n(t, i), C_\infty(t, i)$	state of cell i at time t on the line of length n , on the half-line (time runs downwards in all figures)
minimal-time solution	a rule that synchronizes the line of length n at time $2n - 2$ for every $n \geq 2$
$\mathcal{N}$ -minimal	synchronizes the line of length n at time $2n - 2$ for every $n \in \mathcal{N}$
anti-diagonal M	the cells (t, i) with $t + i = M$
front f_i	$C_\infty(i - 1, i)$, the edge of the light cone
R_n	reflected triangle: the cells of C_n with $t + i \geq 2n - 1$, $t \leq 2n - 2$
$J_n, \text{cone}_n(\tau, \kappa)$	input word of R_n (anti-diagonals $2n - 3$ and $2n - 2$) and the input cells that a cell of R_n depends on
$\text{PUMP}(n, n')$	the pumping condition of Lemma 3.3
chain $c, v_c(i)$	the anti-diagonal $t + i = 2n - 2 + c$ of the line of length n and its cell i
eventually regular	from some time on, finitely many periodic regions separated by signals
germ, $c_M(\delta)$	the transitions used by C_∞ below anti-diagonal M , and their number (without (L, L, L))
$\text{MT}_k(\mathcal{N}; M, N')$	the propositional formula of Section 4
LRAT	the proof format of all certified unsatisfiability claims
$\delta_{12}, \dots, \delta_{14}^e$	the five-state rules of Section 6 and Appendix B

1. Introduction

The firing squad synchronization problem (FSSP), posed by Myhill in 1957 and popularised by Moore [1], asks for a finite automaton that, placed in every cell of a line of arbitrary length n and started with all cells quiescent except a *general* at one end, drives all cells into a common *firing* state for the first time at the same instant (Figure 1). The difficulty is that the cells are identical, that each of them only sees its two neighbours, and that none of them can count up to n : with k states a cell can tell only k situations apart, yet the same rule has to work for every length.

Goto [2] and Waksman [3] showed that the problem can be solved in time $2n - 2$, which is optimal (Remark 2.4). Since then the minimum number of states of a *minimal-time* solution has been a benchmark question in cellular automata theory [4, 5]: Waksman used 16 states, Balzer 8 [6], Gerken 7 [7] and Mazoyer 6 [8]. On the other side, Balzer [6] reported, and Sanders [9] confirmed with a corrected exhaustive search, that no 4-state minimal-time solution exists (Figure 2). Whether five states suffice has remained open since Mazoyer’s construction; the question is recorded as open in the surveys [5, 10]. Many six-state minimal-time solutions are known today [11, 12]; solutions with at most five states are known for restricted families of lengths, for instance the powers of two, some of them in non-minimal time [13, 14],

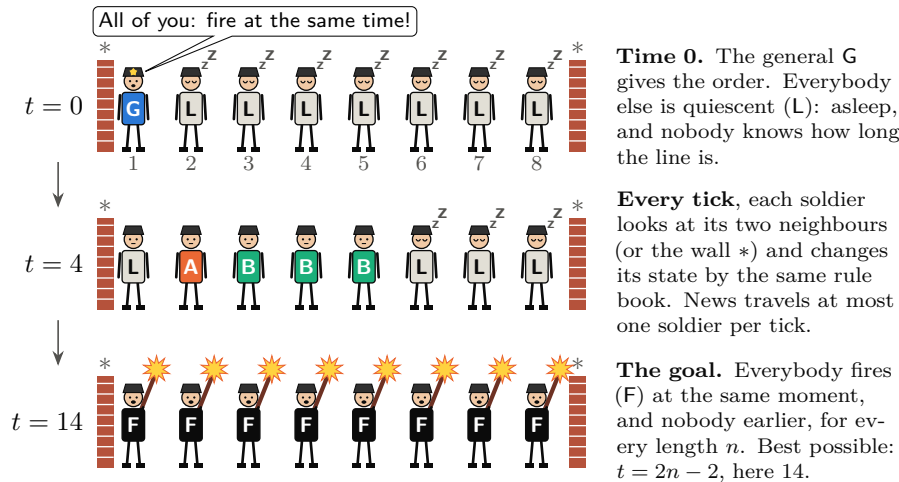


Figure 1: The problem, for a squad of $n = 8$. Top: at time 0 the general (G) gives the order and everybody else is quiescent (L). Middle: at time 4 the news has reached cell 5, and cells 6 to 8 are still asleep. Bottom: at time $2n - 2 = 14$ all soldiers fire together. The three rows are configurations of the five-state rule δ_{14} of Section 6 on the line of length 8.

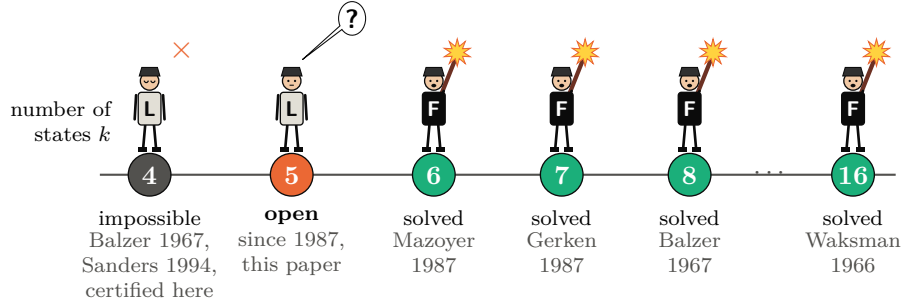


Figure 2: Minimal-time solutions by number of states. Four states are impossible (re-proved with a certificate in Theorem 5.1); five states are the subject of this paper.

and four-state symmetric protocols synchronize the rings whose lengths are powers of two [15]. But no five-state minimal-time solution has been found, nor has its existence been refuted.

The idea in brief. Until the news of its right border comes back, the line of length n cannot tell that it is finite: it behaves exactly like an infinite half-line (Figure 4). A solution therefore consists of a single space-time diagram of the half-line, shared by all lengths, and, for each length n , a triangle near the right end in which the echo of the border is processed. Everything that the triangle of length n knows about the half-line is written on two anti-diagonals. This separation lets us derive necessary conditions on the half-line alone, which are cheap to test, and formulate the search for rules as a satisfiability problem whose negative answers come with independently checkable certificates.

This paper. We do not settle the five-state question. We develop tools that apply to minimal-time solutions with any number of states, use them to give a certified proof of the four-state lower bound that is produced and checked within a minute, and measure precisely how far the same tools reach for five states.

- (i) *Agreement and reflection* (Section 2). The line of length n coincides with the half-line below the anti-diagonal $t + i = 2n - 2$; hence all lines are described by a single half-line diagram C_∞ plus one *reflected triangle* R_n per length, and R_n is a function of two anti-diagonals of C_∞ only (Lemmas 2.2 and 3.1).
- (ii) *Two barriers* (Section 3). In every minimal-time solution the input words of the reflected triangles of any two lengths $4 \leq n < n'$ differ on

an explicit cone of about $n/2$ cells (Lemma 3.3), a constraint on the half-line alone. It implies that the non-periodic part of the half-line reaches the line $i = t/3$ (Theorem 3.4), which explains a priori the speed-1/3 signal of the classical constructions. At the other end, the first chains of the reflected triangle form a finite-state system that must fire the cells near the left border exactly when it reaches them; hence the input anti-diagonals cannot be periodic over long stretches near the left border (Lemma 3.5), the half-line cannot be periodic in any wedge at the border (Theorem 3.6), and in particular it is never eventually regular (Corollary 3.8).

- (iii) *A validated encoding and a certified four-state bound* (Sections 4 and 5). We encode the bounded problem, together with the consequences above, into propositional logic, validate the encoding against Mazoyer’s solution extracted mechanically from Duprat’s Coq formalization [16], and obtain an LRAT certificate, checked in 6 seconds, that no four-state minimal-time solution exists (Theorem 5.1), even without the left quiescence condition.
- (iv) *The five-state frontier* (Section 6). Explicit five-state rules satisfy all our half-line conditions and synchronize every line of length at most 13 (Theorem 6.1), but their common half-line is excluded by Lemma 3.5 (Corollary 6.2). A rule with a chaotic half-line, δ_{14} , synchronizes every line of length at most 14 (Proposition 6.3), and so do four further rules; so every refutation along these lines must use lines of length at least 15. Combining both lemmas with certified solver calls, every five-state solution uses at least 15 distinct neighbourhoods on its half-line below anti-diagonal 78 (Proposition 6.4); of the 9,787 half-lines with at most 16 neighbourhoods, only 16 irregular ones survive.

Table 1 lists the results with the kind of evidence behind each of them.

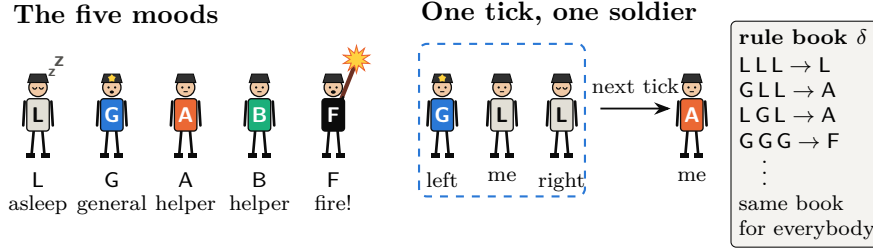
Related work. Surveys of the FSSP are [4, 5]; a classical textbook exposition is [17]. Mazoyer [18] studied the structure of optimal solutions and the information flow they require; Settle and Simon [19] obtained small non-minimal-time solutions. Beyond Balzer and Sanders, lower bounds on the number of states are known for variants: no three-state solution and no four-state symmetric minimal-time solution exist for rings [20]; for further restricted classes see [10]. Computer searches have been used to find new solutions [11] and to generate families of solutions by local simulations [12]. SAT solvers have been applied to many extremal problems in combinatorics

result	where	evidence
agreement, return chain, determinacy	2.2, 2.3, 3.1	proof
pumping lemma; speed-1/3 barrier	3.3, 3.4	proof
left-border lemma and barrier; no eventually regular half-line	3.5–3.8	proof
regularity certificate for concrete half-lines	3.9	proof
mirrored left-border lemma	3.7	proof
no four-state minimal-time solution	5.1	certified computation
δ_{12} , δ_{13} : all lengths up to 12, 13	6.1	simulation
the half-line of δ_{12} , δ_{13} cannot be completed	6.2	proof and simulation
δ_{14} and four more rules: all lengths up to 14	6.3, B.1	simulation
$c_{78} \geq 15$ for every five-state solution	6.4	proof and certified computation
germs of complexity 16 and 17	§6.3, B.2, B.3	certified computation, solver runs
lengths up to 15 for the complexity-15 half-line	B.4	undecided
size of the search trees	§6.4	estimate
chaotic completions should run out near $n \approx 20$	§7	heuristic

Table 1: The results of the paper and the evidence for them. Numbers refer to numbered statements (lemmas, theorems, propositions, corollaries and remarks share one counter per section), B.1–B.4 to the parts of Appendix B. *Simulation*: a finite computation that the programs of Appendix D repeat; *certified computation*: unsatisfiability claims with LRAT proofs checked by `lrat-check`; *solver runs*: solver results without certificates.

with certified unsatisfiability proofs [21, 22]; we are not aware of previous SAT-based work on the state complexity of the FSSP.

Organisation. Section 2 states the problem and reduces all lines to one half-line and a family of reflected triangles. Section 3 proves the two barriers. Section 4 gives the propositional encoding, Section 5 the certified four-state bound and Section 6 the five-state frontier; Section 7 discusses what separates five states from six and lists open problems. Appendix A is a glossary of all terms and symbols, Appendix B records the further experiments, Appendix C gives the rule tables and Appendix D explains how to reproduce every computation. Readers mainly interested in the results can read Sections 1 and 2, Sections 3.1 to 3.3 and Section 6; the computations of Sections 4 to 6 use the lemmas of Section 3 only through their statements.



With five states the book has 94 free lines with 5 possible entries each: $5^{94} \approx 5 \cdot 10^{65}$ books. With four states there are $4^{43} \approx 8 \cdot 10^{25}$ books, and none of them is a minimal-time solution (Theorem 5.1). Is one of the five-state books a minimal-time solution? Nobody knows.

Figure 3: Left: the five states of a five-state rule, drawn as soldiers. Right: one step of one cell. The cell in the middle sees its left neighbour in state G, itself in state L and its right neighbour in state L, looks up the entry GLL of the rule and becomes A. The four entries shown belong to the rule δ_{14} of Table C.10.

2. Preliminaries

This section fixes the model and proves the observation on which the rest of the paper rests: the line of length n agrees with the infinite half-line until the echo of its right border returns, and the influence of the border is confined to a triangle near the right end.

2.1. The problem

Let S be a finite set of *states* containing three distinguished elements: the *quiescent* state L, the *general* G and the *firing* state F; these are pairwise distinct. We write $W := S \setminus \{F\}$ for the set of *working* states and use a symbol $* \notin S$ for the border. A *rule* is a map

$$\delta : (W \cup \{*\}) \times W \times (W \cup \{*\}) \longrightarrow S$$

satisfying the quiescence conditions

$$\delta(L, L, L) = \delta(L, L, *) = L. \quad (1)$$

(Transitions out of F and transitions whose neighbourhood contains F play no role below and are left unspecified; we do *not* require $\delta(*, L, L) = L$, which only strengthens the non-existence statements.) Figure 3 shows the states of a five-state rule and one step of one cell.

For $n \geq 1$ the *line of length n* is the space-time diagram $C_n : \mathbb{N} \times \{1, \dots, n\} \rightarrow S$ defined by $C_n(0, 1) = G$, $C_n(0, i) = L$ for $2 \leq i \leq n$, and

$$C_n(t+1, i) = \delta(C_n(t, i-1), C_n(t, i), C_n(t, i+1)),$$

with the convention $C_n(t, 0) = C_n(t, n+1) = *$; the diagram is only considered up to the first time at which some cell is in state F. The rule *synchronizes the line of length n at time T* if $C_n(T, i) = F$ for all $1 \leq i \leq n$ and $C_n(t, i) \neq F$ for all $t < T$ and all i .

It is classical [1, 3, 6] that a rule which synchronizes *every* line cannot synchronize the line of length $n \geq 2$ before time $2n - 2$ (see Remark 2.4). A *minimal-time solution* is a rule that synchronizes the line of length n at time exactly $2n - 2$ for every $n \geq 2$. (For $n = 1$ one usually asks $\delta(*, G, *) = F$; this transition is used by no other line and is irrelevant here.) A *k -state solution* is one with $|S| = k$.

More generally, for a set $\mathcal{N} \subseteq \{2, 3, \dots\}$ of lengths, a rule is *$\mathcal{N}$ -minimal* if it synchronizes the line of length n at time $2n - 2$ for every $n \in \mathcal{N}$. A minimal-time solution is $\mathcal{N}$ -minimal for every $\mathcal{N}$. Since there are finitely many rules with k states, k -state minimal-time solutions do not exist if and only if there is a finite N such that no k -state rule is $\{2, \dots, N\}$ -minimal; a non-existence proof may therefore consist of a finite family of necessary conditions that no k -state rule satisfies. This is how we prove the four-state bound (Theorem 5.1); for five states we study how large such a family must be.

2.2. The half-line and the agreement lemma

Let $C_\infty : \mathbb{N} \times \{1, 2, \dots\} \rightarrow S$ be the diagram of the half-line: $C_\infty(0, 1) = G$, $C_\infty(0, i) = L$ for $i \geq 2$, and the same recurrence with a border on the left only.

Lemma 2.1 (light cone). *For every rule, every n and every (t, i) with $t < i - 1$ we have $C_n(t, i) = C_\infty(t, i) = L$ (for C_n , whenever $i \leq n$).*

Proof. By induction on t . For $t = 0$ this is the initial condition. Let $t \geq 1$ and $t < i - 1$. The three cells $(t-1, i-1), (t-1, i), (t-1, i+1)$ satisfy $t-1 < (i-1) - 1$, hence are in state L by induction, except that the right neighbour of cell n of C_n is the border. By (1) the new state is L in both cases. $\square$

Lemma 2.2 (agreement). *For every rule and every $n \geq 2$: $C_n(t, i) = C_\infty(t, i)$ whenever $1 \leq i \leq n$ and $t + i \leq 2n - 2$.*

Proof. Induction on t ; the case $t = 0$ is clear. Let $t \geq 1$ and $t + i \leq 2n - 2$. If $i = n$ then $t \leq n - 2 < i - 1$ and both sides equal $\mathbf{L}$ by Lemma 2.1. If $i < n$, the three neighbours $(t - 1, j)$, $j \in \{i - 1, i, i + 1\}$, satisfy $(t - 1) + j \leq t + i \leq 2n - 2$ and $j \leq n$, so they agree by induction (for $j = 0$ both are the border). Hence the new states agree. $\square$

Thus the right border of the line of length n is first “felt” at the cell $(n - 1, n)$, and its influence is confined to the *reflected triangle*

$$R_n := \{(t, i) : 1 \leq i \leq n, t + i \geq 2n - 1, t \leq 2n - 2\}.$$

Lemma 2.3 (return chain). *Let δ synchronize the lines of lengths n and $n + 1$ at times $2n - 2$ and $2n$ respectively. Then*

$$C_n(2n - 1 - i, i) \neq C_\infty(2n - 1 - i, i) \quad (1 \leq i \leq n).$$

*In particular the front $f_i := C_\infty(i - 1, i)$ satisfies $f_{n-1} \neq \mathbf{L}$, and $\delta(f_{n-1}, \mathbf{L}, *) \neq \delta(f_{n-1}, \mathbf{L}, \mathbf{L})$.*

Proof. For $i = 1$: $C_n(2n - 2, 1) = \mathbf{F}$ while, by Lemma 2.2 applied to the line of length $n + 1$ (note $(2n - 2) + 1 \leq 2(n + 1) - 2$), $C_\infty(2n - 2, 1) = C_{n+1}(2n - 2, 1) \neq \mathbf{F}$ because $2n - 2 < 2n$. Suppose the claim holds for some $i < n$. The cell $(2n - 1 - i, i)$ is computed from the cells $(2n - 2 - i, j)$, $j = i - 1, i, i + 1$, whose coordinate sums are $2n - 3$, $2n - 2$ and $2n - 1$. By Lemma 2.2 the first two are the same in C_n and C_∞ (for $i = 1$ the left neighbour is the border in both diagrams). Since the results differ, the right neighbours differ: $C_n(2n - 2 - i, i + 1) \neq C_\infty(2n - 2 - i, i + 1)$, which is the claim for $i + 1$. For $i = n$ we get $C_n(n - 1, n) = \delta(f_{n-1}, \mathbf{L}, *) \neq C_\infty(n - 1, n) = \delta(f_{n-1}, \mathbf{L}, \mathbf{L}) = f_n$, which is impossible if $f_{n-1} = \mathbf{L}$ by (1). $\square$

Remark 2.4. Suppose a rule synchronizes every line, the line of length n at some time $T \leq 2n - 3$. By Lemma 2.2, $C_\infty(T, 1) = C_n(T, 1) = \mathbf{F}$, hence $C_{n'}(T, 1) = \mathbf{F}$ for every $n' \geq n$. For $n' > T + 1$ the cell n' is still quiescent at time T by Lemma 2.1, so the line of length n' has a firing cell at time T without firing entirely: a contradiction.

Figure 4 illustrates Lemma 2.2 and Remark 2.4. The right border only acts after the front has reached it at time $n - 1$, and its influence travels back at speed one; below the anti-diagonal $t + i = 2n - 2$ the line of length n cannot be told apart from any longer line, and the first cell learns where the line ends exactly at time $2n - 2$.

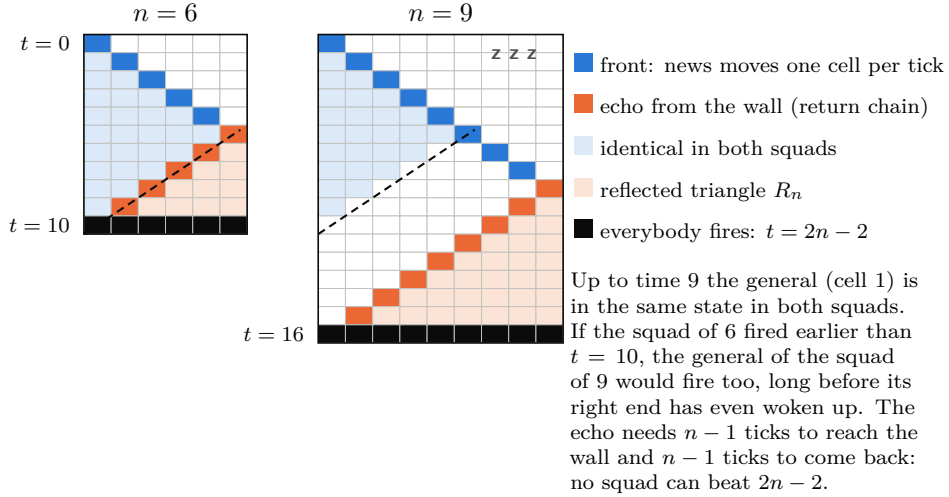


Figure 4: Space-time diagrams (time runs downwards, one square per cell and time step) of the lines of lengths 6 and 9, coloured by what each cell can know. On the cells with $t + i \leq 10$ (light blue, bounded by the dashed anti-diagonal) the two diagrams coincide for every rule (Lemma 2.2); the right border acts only in the reflected triangle R_n , below the return chain. The colours show regions, not states.

Applying Lemma 2.3 to all n shows that in a minimal-time solution every cell i leaves L exactly at time $i - 1$ (the front moves at speed one) and, since $C_\infty(t, i) = C_n(t, i)$ for n large,

$$C_\infty(t, i) \neq \mathsf{F} \quad \text{for all } t \geq 0, i \geq 1. \quad (2)$$

3. Two barriers on the half-line

A minimal-time solution has to fire every line at exactly the right moment, for all lengths at once. This section turns that requirement into two necessary conditions on the half-line diagram C_∞ alone, one for each end of the line. At the right end, the firing of the last cell of the line of length n is decided by the right half of the two anti-diagonals on which the line stops agreeing with the half-line; the lines of two different lengths must therefore never look alike there (the pumping lemma, Lemma 3.3), and this forces the half-line to keep changing along the line $i = t/3$ (Theorem 3.4). At the left end, the cells near the border are fired by a narrow band of anti-diagonals that behaves like a finite automaton reading the half-line; such an automaton cannot count along a long periodic stretch (the left-border lemma, Lemma 3.5), so the

half-line cannot be periodic near the border (Theorem 3.6 and Corollary 3.8). Figures 5 and 6 show the objects involved.

3.1. Reflected triangles in relative coordinates

Fix $n \geq 2$. For a cell (t, i) of the line of length n put $\tau := t - (n - 1)$ (time elapsed since the front reached the right end) and $\kappa := n - i$ (distance to the right end). Then

$$R_n = \{(\tau, \kappa) : 0 \leq \kappa \leq \tau \leq n - 1\},$$

the right border sits at $\kappa = -1$ and the left border at $\kappa = n$. A cell $(\tau, \kappa) \in R_n$ is computed from $(\tau - 1, \kappa + 1), (\tau - 1, \kappa), (\tau - 1, \kappa - 1)$; each of these is either in R_n , or a border, or satisfies $\tau' - \kappa' \in \{-1, -2\}$, i.e. lies on one of the anti-diagonals $t + i = 2n - 2, t + i = 2n - 3$, where C_n coincides with C_∞ by Lemma 2.2. We call

$$J_n(\kappa) := (C_\infty(n - 3 + \kappa, n - \kappa), C_\infty(n - 2 + \kappa, n - \kappa)), \quad 0 \leq \kappa \leq n - 1,$$

the *input word* of R_n ; its two components are the *lower* ($t + i = 2n - 3$) and *upper* ($t + i = 2n - 2$) input cells. (For $n = 2$ and $\kappa = 0$ the lower cell would lie at time -1 ; it is a predecessor of no cell of R_n , and we set it to L.) Figure 5 shows these regions for $n = 12$.

Lemma 3.1 (determinacy). *There is a single partial map Ψ , depending on δ only, such that for every $n \geq 2$ and every $(\tau, \kappa) \in R_n$, $C_n(n - 1 + \tau, n - \kappa)$ is obtained by evaluating Ψ at (τ, κ) on the input word J_n with borders at $\kappa = -1$ and $\kappa = n$. In particular the value only depends on the input cells and borders in the backward dependency cone of (τ, κ) .*

Proof. The recurrence in relative coordinates does not mention n ; only the position of the left border does. $\square$

For $(\tau, \kappa) \in R_n$ let $\text{cone}_n(\tau, \kappa)$ be the set of input cells, i.e. of pairs consisting of a position κ' and one of the two anti-diagonals, that are reached from (τ, κ) by repeatedly passing to the three predecessors, stopping at input cells and borders; say that the cone is *right-anchored* if it does not reach the left border $\kappa' = n$.

Lemma 3.2. *For $n \geq 4$ the cone of the right end at the firing time, $\text{cone}_n(n - 1, 0)$, is right-anchored and consists of the upper input cell at $\kappa' = 0$, both input cells at $1 \leq \kappa' \leq \lfloor n/2 \rfloor$, and, when n is odd, the lower input cell at $\kappa' = (n + 1)/2$.*

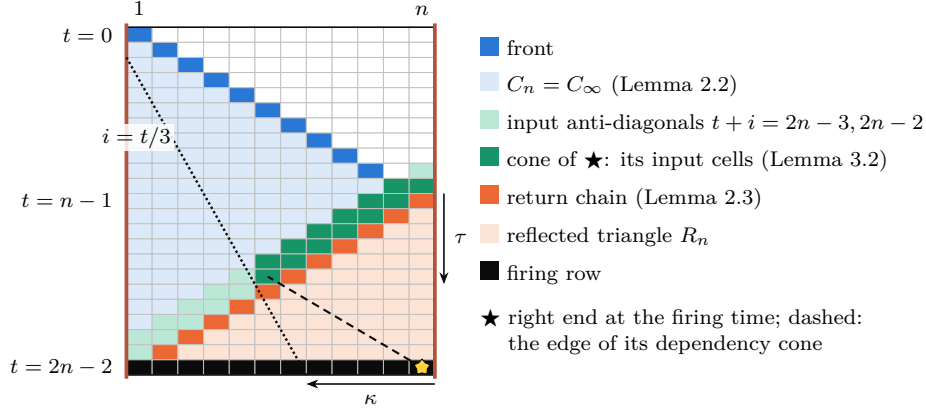


Figure 5: Anatomy of the line of length $n = 12$ (time runs downwards, one square per cell and time step; the colours show regions, not states). Above the two input anti-diagonals the line coincides with the half-line (Lemma 2.2); the reflected triangle R_n below them is computed from these two anti-diagonals alone (Lemma 3.1), in the relative coordinates τ and κ measured from the right end. The firing of the right end ($\star$) depends only on the dark input cells, its cone (Lemma 3.2), which ends where the dotted line $i = t/3$ crosses the input anti-diagonals (Theorem 3.4).

Proof. A predecessor step changes τ by -1 and κ by at most one, and input cells satisfy $\tau' = \kappa' - 1$ or $\kappa' - 2$. Starting from $(n-1, 0)$, a path that reaches an input cell at position κ' after s steps satisfies $\kappa' \leq s$ and $n-1-s = \kappa' - 1$ or $\kappa' - 2$; hence $2\kappa' \leq n+1$ (upper cells: $2\kappa' \leq n$). Conversely, the upper cell at a position κ' with $2\kappa' \leq n$ is reached by moving left κ' times and then staying put, and the lower cell at a position $\kappa' \geq 1$ with $2\kappa' \leq n+1$ by moving left $\kappa' - 1$ times, staying put down to the cell $(\kappa' - 1, \kappa' - 1)$ and moving left once more; all intermediate cells satisfy $\tau \geq \kappa$ and lie in R_n . The lower cell at $\kappa' = 0$ is a predecessor of no cell of R_n . The largest position reached is $\lceil n/2 \rceil \leq n-1$, and a path can reach $\kappa = n$ only after n left moves, i.e. at a negative time, so the left border is never reached; the right border only contributes the constant symbol $*$. (The same set is also computed mechanically in the accompanying code, file `pumping.py`.) $\square$

3.2. The pumping lemma

The right end of the line of length n fires at the relative time $\tau = n-1$, and by Lemmas 3.1 and 3.2 this event is decided by the input cells of its cone, the dark cells of Figure 5: roughly the right halves of the two input anti-diagonals. A longer line, of length $n' > n$, reaches the same relative cell

$(\tau, \kappa) = (n - 1, 0)$ at the time $n' + n - 2$, before its own firing time $2n' - 2$. If its input word agreed with that of the line of length n on the whole cone, its right end would fire at that moment, too early. Hence:

Lemma 3.3 (pumping). *Let δ be a minimal-time solution and $4 \leq n < n'$. Then J_n and $J_{n'}$ differ on $\text{cone}_n(n - 1, 0)$.*

Proof. Suppose they agree there. By Lemmas 3.1 and 3.2 the relative cell $(n - 1, 0)$ of $R_{n'}$ (which belongs to $R_{n'}$ since $n - 1 \leq n' - 1$, and whose cone is the same set of input positions, far from the left border $\kappa = n' > n$) has the same value as the relative cell $(n - 1, 0)$ of R_n , namely $C_n(2n - 2, n) = \mathbf{F}$. Thus $C_{n'}(n' + n - 2, n') = \mathbf{F}$ although $n' + n - 2 < 2n' - 2$, contradicting minimality for n' . $\square$

The point of Lemma 3.3 is that it constrains the *half-line diagram only*: no reflected triangle has to be computed. It is the source of the constraint family $\text{PUMP}(n, n')$ used in Section 4. As a sanity check, we verified mechanically that Mazoyer's six-state solution satisfies $\text{PUMP}(n, n')$ for all $4 \leq n < n' \leq 120$.

3.3. A speed-1/3 barrier

Write the half-line diagram in the frame of the front: $\Phi_t(j) := C_\infty(t, t + 1 - j)$ for $0 \leq j \leq t$ (the cell at *depth* j behind the front at time t). By the recurrence,

$$\Phi_{t+1}(j) = \delta(\Phi_t(j), \Phi_t(j - 1), \Phi_t(j - 2)) \quad (0 \leq j \leq t), \quad (3)$$

with $\Phi_t(-1) = \Phi_t(-2) = \mathbf{L}$: in this frame the half-line is a *one-sided* automaton in which information only travels to larger depths, at most two depths per step. In particular every depth-row $t \mapsto \Phi_t(j)$ is eventually periodic. In these coordinates $J_n(\kappa) = (\Phi_{n-3+\kappa}(2\kappa - 2), \Phi_{n-2+\kappa}(2\kappa - 1))$.

The pumping lemma therefore says that the depth-rows cannot all become periodic early: if every input cell of the cone were read in the periodic regime of its depth-row, the lines of lengths n and $n + P$, for a common period P , would have the same input word on the cone. The far end of the cone lies at depth about n and at time about $3n/2$ (Figure 5), and this gives the following bound.

Theorem 3.4 (speed-1/3 barrier). *Let δ be a minimal-time solution. Every depth-row $t \mapsto \Phi_t(j)$ is eventually periodic; let $\theta(j)$ be the time from which it is periodic. Then for every $n \geq 4$ there is a depth $j \leq n$ with $\theta(j) > n + \frac{j}{2} - \frac{5}{2}$. Consequently $\limsup_{j \rightarrow \infty} \theta(j)/j \geq 3/2$: along infinitely many depth-rows the*

transient part extends, up to an arbitrarily small angle, to the line $i = t/3$ of the original coordinates.

Proof. Row 0 satisfies $\Phi_{t+1}(0) = \delta(\Phi_t(0), L, L)$ and is eventually periodic; by (3), row j is a finite-state system driven by the eventually periodic rows $j-1$, $j-2$, hence eventually periodic, with some period P_j . Fix $n \geq 4$ and let P be a common multiple of $P_0, \dots, P_{n+1}$. If every input cell of $\text{cone}_n(n-1, 0)$ were read in the periodic regime of its row, i.e. $n-3+\kappa \geq \theta(2\kappa-2)$ and $n-2+\kappa \geq \theta(2\kappa-1)$ for all cone positions κ , then $J_{n+P} = J_n$ on the cone, contradicting Lemma 3.3. Cone positions satisfy $\kappa \leq \lceil n/2 \rceil$, so the violated inequality concerns a depth $j \in \{2\kappa-2, 2\kappa-1\}$ with $j \leq n$ and $\theta(j) > n-3+\kappa \geq n + \frac{j}{2} - \frac{5}{2}$. If $\limsup_{j \rightarrow \infty} \theta(j)/j < 3/2$, there are $\alpha < 3/2$ and β with $\theta(j) \leq \alpha j + \beta$ for all j , and enlarging α we may assume $\alpha \geq \frac{1}{2}$. Then for n large and every $j \leq n$ we would have $(\alpha - \frac{1}{2})j \leq (\alpha - \frac{1}{2})n < n - \beta - \frac{5}{2}$, i.e. $\theta(j) \leq n + \frac{j}{2} - \frac{5}{2}$, a contradiction. Finally a cell of depth j at time $\theta(j)$ sits at position $\theta(j) + 1 - j$, which is at least $\theta(j)/3 + 1$ when $\theta(j) \geq 3j/2$. $\square$

Theorem 3.4 explains a priori why every minimal-time construction contains structure reaching the line $i = t/3$, whose crossing with the anti-diagonal $t + i = 2n - 2$ is, up to one cell, the far end of $\text{cone}_n(n-1, 0)$: the speed-1/3 signal emitted by the general in the constructions of Waksman and Balzer [3, 6] runs along this line, and in Mazoyer’s solution the periodic zone behind the front is bounded by a line of speed 1/2; indeed we checked that for $1 \leq j < 700$ the depth-row j is 3-periodic from time $\theta(j) = 2j + 2$ when j is even or $j = 1$, and from $\theta(j) = 2j + 1$ when $j \geq 3$ is odd. The theorem also explains the failure of many of the “lucky” partial rules found by the solvers (Section 6): their half-line diagrams are periodic behind the front far beyond the line $i = t/3$.

3.4. A second barrier, at the left border

Lemma 3.3 looks at the right end of the line. The left end gives a constraint of a different nature, again on the half-line only. The idea is the following. Near the left border, the reflected triangle is produced by a band of consecutive anti-diagonals, the *chains*, that follow the reflected wave. Read from right to left, this band behaves like a finite automaton that moves one cell per step and reads the two input anti-diagonals as it goes (Figure 6), and it must fire the cell $2s$ exactly when it arrives there. An automaton with finitely many states that reads a long periodic stretch of its input returns to an earlier state; it cannot tell where the stretch ends, so it cannot time this firing. Lemma 3.5 makes this quantitative.

To set this up, fix $n \geq 2$ and number the anti-diagonals of the line of length n from the input anti-diagonals on: for $c \geq -1$ and $1 \leq i \leq n$ with $c \leq i$ put

$$v_c(i) := C_n(2n - 2 + c - i, i),$$

the cell of the *chain* $t + i = 2n - 2 + c$ at position i . Chains -1 and 0 are the input anti-diagonals, on which $v_c(i) = C_\infty(2n - 2 + c - i, i)$ by Lemma 2.2; chains $c \geq 1$ lie in R_n , and chain 1 is the return chain of Lemma 2.3. The cell $(2n - 2 + c - i, i)$ has its three predecessors on the chains $c - 2$, $c - 1$ and c , so

$$v_c(i) = \delta(v_{c-2}(i-1), v_{c-1}(i), v_c(i+1)) \quad (c \geq 1), \quad (4)$$

where $v_{c-2}(0)$ and $v_c(n+1)$ stand for the border. If the line of length n is synchronized at time $2n - 2$, then $v_i(i) = \mathbf{F}$ for $1 \leq i \leq n$ and $v_c(i) \neq \mathbf{F}$ for $-1 \leq c < i$. In the skewed coordinates $u_c(y) := v_c(y + \lfloor c/2 \rfloor)$, (4) becomes

$$u_c(y) = \delta(u_{c-2}(y), u_{c-1}(y + \varepsilon_c), u_c(y+1)), \quad \varepsilon_c = \begin{cases} 1, & c \text{ even,} \\ 0, & c \text{ odd,} \end{cases} \quad (5)$$

with $u_0(y) = C_\infty(2n - 2 - y, y)$ and $u_{-1}(y) = C_\infty(2n - 2 - y, y - 1)$, and $u_c(y)$ is the cell $y + \lfloor c/2 \rfloor$ at time $2n - 2 + \lceil c/2 \rceil - y$: it must be $\mathbf{F}$ if $\lceil c/2 \rceil = y$ and must differ from $\mathbf{F}$ if $\lceil c/2 \rceil < y$. Thus each *slice* $(u_1(y), u_2(y), \dots)$ is obtained from the slice $y + 1$ by a sequential finite-state transducer that reads the components in increasing order and whose initial state is determined by the two input cells $u_{-1}(y), u_0(y)$ (Figure 6): read from right to left, the reflected triangle is a cascade of finite-state transducers, the left-hand counterpart of the co-moving description (3) of the half-line.

Lemma 3.5 (left border). *Let δ have k states and synchronize the line of length n at time $2n - 2$. Let $s \geq 1$, $L \geq 1$ and $s + L \leq Y \leq n - s$, and suppose that the words*

$$\begin{aligned} y &\mapsto C_\infty(2n - 2 - y, y) & (s \leq y \leq Y) & \quad \text{and} \\ y &\mapsto C_\infty(2n - 3 - y, y) & (s \leq y \leq Y - 1) \end{aligned}$$

are L -periodic. Then $Y - s \leq (k - 1)^{2s} L$.

Proof. For $s \leq y \leq Y$ let $\mathbf{u}(y) := (u_1(y), \dots, u_{2s}(y))$. These are cells $y + \lfloor c/2 \rfloor \leq Y + s \leq n$ with $\lceil c/2 \rceil \leq s \leq y$, so they are defined, and for $y > s$ none of them is $\mathbf{F}$. By (5), $u_c(y)$ with $1 \leq c \leq 2s$ is computed from components of index $< c$ of the same slice, from components of index $\leq c$ of

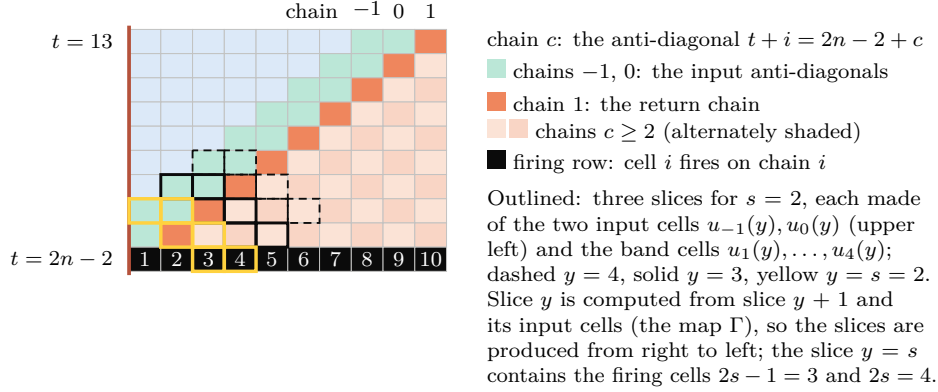


Figure 6: The lower left corner of the line of length $n = 12$ (time runs downwards; the colours show regions, not states), with the chains of the left-border lemma and three of the slices used in its proof.

slice $y + 1$, and from $u_{-1}(y), u_0(y)$; hence there is a map Γ , depending on δ only, with

$$\mathbf{u}(y) = \Gamma(\mathbf{u}(y + 1), u_{-1}(y), u_0(y)) \quad (s + 1 \leq y \leq Y - 1).$$

Unfolding (5) for even c , $u_{2j}(y) = \delta(u_{2j-2}(y), u_{2j-1}(y + 1), u_{2j}(y + 1))$, shows that $u_{2s}(y)$ depends only on $u_0(y)$ and $\mathbf{u}(y + 1)$: there is a map Γ' with $u_{2s}(y) = \Gamma'(\mathbf{u}(y + 1), u_0(y))$ whenever $s \leq y \leq Y - 1$. By hypothesis, u_0 is L -periodic on $[s, Y]$ and u_{-1} on $[s + 1, Y]$.

Suppose $Y - s > (k - 1)^{2s}L$. The pairs $(\mathbf{u}(y), y \bmod L)$, $s + 1 \leq y \leq Y$, take at most $(k - 1)^{2s}L$ values, so there are $s + 1 \leq y_1 < y_2 \leq Y$ with $\mathbf{u}(y_1) = \mathbf{u}(y_2)$ and $y_1 \equiv y_2 \pmod{L}$; put $h := y_2 - y_1$. By downward induction, $\mathbf{u}(y) = \mathbf{u}(y + h)$ for $s + 1 \leq y \leq y_1$: this holds for $y = y_1$, and if it holds for $y + 1 \leq y_1$ then $\mathbf{u}(y) = \Gamma(\mathbf{u}(y + 1), u_{-1}(y), u_0(y)) = \Gamma(\mathbf{u}(y + 1 + h), u_{-1}(y + h), u_0(y + h)) = \mathbf{u}(y + h)$, because $y + h < y_2 \leq Y$ and $L \mid h$. In particular $\mathbf{u}(s + 1) = \mathbf{u}(s + 1 + h)$, and since $s + h \leq Y - 1$ and $u_0(s + h) = u_0(s)$,

$$u_{2s}(s + h) = \Gamma'(\mathbf{u}(s + 1 + h), u_0(s + h)) = \Gamma'(\mathbf{u}(s + 1), u_0(s)) = u_{2s}(s) = F.$$

But $\lceil 2s/2 \rceil = s < s + h$, so $u_{2s}(s + h)$, the cell $2s + h$ at time $2n - 2 - h$, is not F : a contradiction. $\square$

The border itself is harmless (cell 1 fires because the return chain meets the border), which is why the lemma starts at $s \geq 1$ and pays a factor exponential in s : the band of the first $2s$ chains has up to $(k - 1)^{2s}$ states.

Theorem 3.6 (left-border barrier). *Let δ be a minimal-time solution. There are no $s \geq 1$, $\alpha > 0$, t_0 and periods $p, q \geq 1$ such that $C_\infty(t+p, i) = C_\infty(t, i)$ and $C_\infty(t, i+q) = C_\infty(t, i)$ whenever the cells involved lie in the wedge $\Omega := \{(t, i) : s \leq i \leq \alpha t, t \geq t_0\}$.*

Proof. Suppose the contrary and put $L := pq$. If (t, i) and $(t-L, i+L)$ lie in Ω , then so do the intermediate cells $(t-L, i+jq)$ and $(t-L+jp, i)$, and $C_\infty(t-L, i+L) = C_\infty(t-L, i) = C_\infty(t, i)$. For the anti-diagonals $t+i = 2n-2$ and $t+i = 2n-3$ this makes the words of Lemma 3.5 L -periodic on $[s, Y_n]$ for some Y_n with $Y_n \geq \frac{\alpha}{1+\alpha} 2n - \gamma$, where γ depends on α, L, t_0 only. For n large, $\min(Y_n, n-s) - s > (k-1)^{2s}L$, contradicting Lemma 3.5. $\square$

Remark 3.7 (the right end). The proof of Lemma 3.5 uses neither Lemma 2.2 nor the position of the chains inside R_n : it only uses the recurrence (4) for $c \geq 1$, the firing condition ($v_c(i) = \mathbf{F}$ if and only if $c = i$) and the periodicity of the two chains $c = -1, 0$. It therefore applies verbatim to the mirror image of the line, i.e. to the rule $\bar{\delta}(x_1, x_2, x_3) := \delta(x_3, x_2, x_1)$ and the cells $\bar{i} := n+1-i$, whose chains c are the diagonals $t-i = n-3+c$ of the line of length n . Hence, if δ synchronizes the line of length n at time $2n-2$, the words

$$\begin{aligned} y &\mapsto C_n(2n-2-y, n+1-y) & (s \leq y \leq Y), \\ y &\mapsto C_n(2n-3-y, n+1-y) & (s \leq y \leq Y-1), \end{aligned}$$

read along the diagonals $t-i = n-3$ and $t-i = n-4$ from the right end, are not L -periodic when $Y-s > (k-1)^{2s}L$ and $Y \leq n-s$. For $y \leq n/2$ these cells lie in R_n : seen from the right end, the reflected triangles cannot be periodic over long stretches either. Unlike Lemma 3.5, this is a constraint on the reflected part of the rule, not on the half-line. (On Mazoyer's rule, for the lengths $n \leq 300$ with $n \equiv 6 \pmod{7}$, $s \leq 2$ and $L \leq 6$, these stretches have length at most 7.)

3.5. Eventually regular half-lines

A half-line that settles into finitely many periodic regions separated by signals is periodic in a wedge at the border, so the left-border barrier excludes it. We make this precise and then give a finite test that proves this behaviour for a concrete half-line.

Say that the half-line of a rule is *eventually regular* if there are t_1 , $P \geq 1$ and, for each residue $r \in \{0, \dots, P-1\}$, words $w_0, \dots, w_m$ and nonempty

words $z_1, \dots, z_m$ over W and integers $a_j \geq 0$, $D_j \geq 0$ such that for all $\nu \geq 0$ the configuration $C_\infty(t, 1) \cdots C_\infty(t, t+1)$ at time $t = t_1 + r + \nu P$ is the word

$$w_0 z_1^{a_1 + \nu D_1} w_1 z_2^{a_2 + \nu D_2} w_2 \cdots z_m^{a_m + \nu D_m} w_m,$$

i.e. from some time on the half-line consists of finitely many periodic regions separated by finitely many signals of rational speeds.

Corollary 3.8. *The half-line of a minimal-time solution is not eventually regular.*

Proof. Suppose it is. For each residue r , some D_j is positive, because the length of the configuration grows by P per period; let $j(r)$ be the least such index. The blocks and junctions to the left of $z_{j(r)}$ have constant lengths, so the block $z_{j(r)}^{a_{j(r)} + \nu D_{j(r)}}$ starts at a fixed cell s_r and its length grows linearly in t , at least like βt for some $\beta > 0$ and t large. Inside it, the state of a cell depends only on its distance to s_r modulo $|z_{j(r)}|$, and it is the same at the times t and $t + P$. Put $s := \max_r s_r$, $q := \prod_r |z_{j(r)}|$ and $\alpha := \beta/2$. For t_0 large, every cell of the wedge $\{(t, i) : s \leq i \leq \alpha t, t \geq t_0\}$ lies in the block of index $j(r)$ of its residue class, so C_∞ is periodic with periods P in time and q in space on this wedge, contradicting Theorem 3.6. $\square$

Corollary 3.8 applies whatever the number of states. On a concrete germ it is used through the following finite test. For words $w_0, \dots, w_m$ and nonempty words $z_1, \dots, z_m$ over W and $\mathbf{e} \in \mathbb{N}^m$ put $X(\mathbf{e}) := w_0 z_1^{e_1} w_1 \cdots z_m^{e_m} w_m$, and let $\mathbf{1}_j$ denote the j th unit vector.

Lemma 3.9 (regularity certificate). *Let δ be a rule, $t_1 \geq 0$ and $P \geq 1$. Suppose that the configuration of the half-line at time t_1 is $X(\mathbf{e})$ with $e_j |z_j| \geq 2P + |z_j|$ for all j , and that the configuration at time $t_1 + P$ is $X(\mathbf{e} + \mathbf{D})$ for some $\mathbf{D} \in \mathbb{N}^m$. Then the configuration at time $t_1 + \nu P$ is $X(\mathbf{e} + \nu \mathbf{D})$ for every $\nu \geq 0$, and C_∞ is periodic in time and space on a wedge at the left border. Consequently no rule that agrees with δ on the neighbourhoods occurring in C_∞ up to time $t_1 + P$ is a minimal-time solution.*

Proof. For a word Y over W let $\Lambda(Y)$ be the word formed by the cells $1, \dots, |Y| + P$ after P steps of the half-line started from Y followed by quiescent cells, with the border on the left; by (1) the cells beyond $|Y| + P$ are still quiescent, and Λ maps the configuration at any time t to the configuration at time $t + P$. We write $Y[p]$ for the letter at position p , with $Y[p] = \mathbf{L}$ for $p > |Y|$. By locality, $\Lambda(Y)[p]$ depends only on the letters of

Y at the positions $p - P, \dots, p + P$ (and on the border if $p \leq P$). Write $q_j := |z_j|$. Since $|\Lambda(Y)| = |Y| + P$, we have $\sum_j D_j q_j = P$.

Let $\mathbf{e}' \geq \mathbf{e}$ (componentwise) with $\Lambda(X(\mathbf{e}')) = X(\mathbf{e}' + \mathbf{D})$, and fix j . Put $Y := X(\mathbf{e}')$, $q := q_j$, and let block j of Y occupy the positions $a, \dots, b$, so that $b - a + 1 = e'_j q \geq 2P + q$. The word $Y' := X(\mathbf{e}' + \mathbf{1}_j)$ satisfies $Y'[p] = Y[p]$ for $p \leq b$ and $Y'[p] = Y[p - q]$ for $p \geq a + q$. By locality, $\Lambda(Y')[p] = \Lambda(Y)[p]$ for $p \leq b - P$ and $\Lambda(Y')[p] = \Lambda(Y)[p - q]$ for $p \geq a + q + P$; since $a + q + P \leq b - P + 1$, every position is covered, so $\Lambda(Y')$ arises from $\Lambda(Y) = X(\mathbf{e}' + \mathbf{D})$ by repeating the factor of length q that ends at position $b - P$. Block j of $X(\mathbf{e}' + \mathbf{D})$ occupies the positions from $a + \sum_{i < j} D_i q_i \leq a + P$ to $b + \sum_{i \leq j} D_i q_i \geq b$, so this factor, which starts at $b - P - q + 1 \geq a + P$, lies inside it, and repeating a factor of length q inside a q -periodic block lengthens the block by q . Hence $\Lambda(X(\mathbf{e}' + \mathbf{1}_j)) = X(\mathbf{e}' + \mathbf{1}_j + \mathbf{D})$. The same comparison, made after each of the P steps, shows that every neighbourhood used in the computation of $\Lambda(Y')$ is used in that of $\Lambda(Y)$.

Starting from $\mathbf{e}' = \mathbf{e}$ and adding one period at a time, we obtain $\Lambda(X(\mathbf{e}')) = X(\mathbf{e}' + \mathbf{D})$ for all $\mathbf{e}' \geq \mathbf{e}$, hence, by induction on ν , that the configuration at time $t_1 + \nu P$ is $X(\mathbf{e} + \nu \mathbf{D})$; all neighbourhoods involved occur in C_∞ up to time $t_1 + P$, so the same holds for every rule that agrees with δ on them.

Let j be the least index with $D_j > 0$ (it exists since $\sum_j D_j q_j = P$), put $q := q_j$, and let s be the first position of block j ; it does not depend on ν because $D_i = 0$ for $i < j$. At time $t_1 + \nu P$ block j occupies the positions $s, \dots, s + \ell_\nu - 1$ with $\ell_\nu = (e_j + \nu D_j)q \geq \nu$, and the configurations at the times $t_1 + \nu P$ and $t_1 + (\nu + 1)P$ coincide on the positions $1, \dots, s + \ell_\nu - 1$ and are q -periodic on $[s, s + \ell_\nu - 1]$. By locality, for $0 \leq r < P$ the configurations at the times $t := t_1 + \nu P + r$ and $t + P$ coincide on $[1, s + \ell_\nu - 1 - P]$ and are q -periodic on $[s + P, s + \ell_\nu - 1 - P]$. Since $\ell_\nu \geq (t - t_1)/P - 1$, C_∞ is periodic with periods P in time and q in space on the wedge $\{(t, i) : s + P \leq i \leq t/(2P), t \geq t_0\}$ for t_0 large, and Theorem 3.6 gives the last statement. $\square$

The program `germpersist.py` searches for such decompositions, with block periods $|z_j|$ up to a bound $q_{\max}$ and blocks of length at least $2P + 2q_{\max}$, at the times $t_1 + r$ for every residue $r = 0, \dots, P - 1$; it thereby shows that the half-line is eventually regular. It also checks the induction step directly, by evolving the configurations with one or two additional periods in each block, and compares the predicted configurations 100 periods later with a direct simulation.

3.6. The barriers in the classical constructions

Theorems 3.4 and 3.6 complement each other: the half-line of a minimal-time solution has non-periodic structure reaching the line $i = t/3$ behind the front, and non-periodic structure in every wedge at the left border. In the constructions of Waksman and Balzer the latter is the family of signals of speeds $1/(2^j - 1)$, which accumulate at the border; in Mazoyer's solution it is the family of recursive division points (Figure 7). The classical halving idea [17] sends from the general a signal of speed 1, the hare, and one of speed $1/3$, the tortoise; the hare is reflected at the right border at time $n - 1$ and meets the tortoise in the middle at time $3(n - 1)/2$, after which both halves are treated in the same way, which takes about $3n$ steps in all. The minimal-time constructions send the signals of all speeds $1/(2^j - 1)$ at once: the reflected signal meets them at the points $n/2, n/4, \dots$ at exactly the times at which the pieces between them can be synchronized recursively so that all pieces fire at time $2n - 2$. The tortoise of speed $1/3$ runs along the line $i = t/3$ of Theorem 3.4, and the slower ones fill every wedge at the border, as Theorem 3.6 requires. We checked Lemma 3.5 mechanically on Mazoyer's rule for $n \leq 400$, $s \leq 3$, $L \leq 6$: its periodic stretches near the left border have length at most 6, far below the bound. Unlike Lemma 3.3, Lemma 3.5 needs long periodic stretches (for five states, more than $16L$ cells when $s = 1$ and more than $256L$ cells when $s = 2$), hence long lines, but it applies to any half-line that can be computed far enough, which we exploit in Section 6.

4. Propositional encoding

Fix k (the number of states), write $W = \{L, G, A_1, \dots, A_{k-3}\}$ (for $k = 5$, $A_1 = A$ and $A_2 = B$) and let $\mathcal{N}$ be a finite set of lengths. We build a CNF formula $\text{MT}_k(\mathcal{N}; M, N')$ that is satisfiable whenever a k -state minimal-time solution exists; here M bounds the half-line and N' the pumping constraints, and we write $\text{MT}_k(\mathcal{N}; M)$ when no pumping constraints are added. The formula transcribes the space-time diagrams directly: there is one propositional variable for each possible entry of the rule table and for each possible state of each relevant cell, and the clauses force the cells to follow the rule and the lines to fire on time. All constraint families are either the definition of $\mathcal{N}$ -minimality or consequences of Lemmas 2.2, 2.3, 3.1 and 3.3 and (2); none relies on unproved assumptions about the shape of solutions. Table 2 lists them; the paragraphs below give the details.

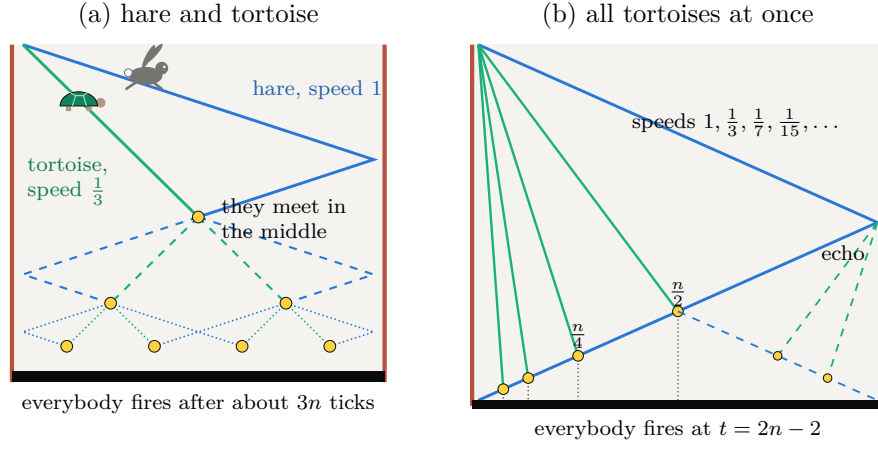


Figure 7: Space-time sketches of two classical constructions (time runs downwards; borders in red). (a) Halving: the hare (speed 1) and the tortoise (speed $1/3$) meet in the middle, and the construction is repeated in both halves, here for $n = 17$. (b) Minimal time, $n = 33$: signals of speeds $1/(2^j - 1)$ meet the reflected signal at $n/2, n/4, n/8, \dots$; each piece is synchronized from one of its ends, as shown with dashes for the right half, and all pieces fire at $t = 2n - 2 = 64$.

family	requirement	justification
half-line	the cells of C_∞ up to the anti-diagonal M follow δ , and none of them fires	definition of C_∞ ; (2)
lines	for $n \in \mathcal{N}$: the cells of R_n follow δ , none of them fires before time $2n - 2$, and all n cells fire at time $2n - 2$	$\mathcal{N}$ -minimality; Lemma 2.2
return chains	$C_n \neq C_\infty$ on the anti-diagonal $t + i = 2n - 1$	Lemma 2.3
pumping	J_n and $J_{n'}$ differ on $\text{cone}_n(n - 1, 0)$ for $4 \leq n < n' \leq N'$	Lemma 3.3
links	equal input words give equal cells in R_n and $R_{n'}$	Lemma 3.1 (implied by the others)
symmetry breaking	the auxiliary states first appear in a fixed order	renaming of the auxiliary states

Table 2: The constraint families of $\text{MT}_k(\mathcal{N}; M, N')$. Return chains, links and symmetry breaking are optional; the pumping constraints are absent when N' is omitted.

Variables. For every neighbourhood $e = (x, y, z) \in (W \cup \{*\}) \times W \times (W \cup \{*\})$ other than $(*, y, *)$, one-hot variables $T_{e,d}$, $d \in S$, encode $\delta(e) = d$; the two quiescence transitions (1) are fixed by unit clauses. For every cell whose

state is not a constant, one-hot variables $X_{\xi,w}$, $w \in W$, encode its state (all cells whose state is used before the firing time are in W).

Transition clauses. For a cell ξ with neighbours ξ_1, ξ_2, ξ_3 (cells, constants L, G, or the border) and for all (x, y, z) in the respective domains and $d \in W$ we add

$$\begin{aligned} &\neg X_{\xi_1,x} \vee \neg X_{\xi_2,y} \vee \neg X_{\xi_3,z} \vee \neg T_{(x,y,z),d} \vee X_{\xi,d}, \\ &\neg X_{\xi_1,x} \vee \neg X_{\xi_2,y} \vee \neg X_{\xi_3,z} \vee \neg T_{(x,y,z),F} \end{aligned}$$

(the second clause forbids firing). For a cell that must fire we add instead $\neg X_{\xi_1,x} \vee \neg X_{\xi_2,y} \vee \neg X_{\xi_3,z} \vee T_{(x,y,z),F}$. Literals of constant neighbours are omitted.

Half-line. The cells (t, i) of C_∞ with $t \geq 1$, $1 \leq i \leq t + 1$ and $t + i \leq M$ receive variables and transition clauses; cells with $i > t + 1$ are the constant L (Lemma 2.1). Every such cell is forbidden to fire, by (2).

Lines. For every $n \in \mathcal{N}$ (with $2n - 2 \leq M$) the cells of R_n with $t \leq 2n - 3$ receive variables; their neighbours outside R_n are the corresponding half-line cells (Lemma 2.2). All of them are forbidden to fire and the n cells at time $2n - 2$ are required to fire.

Return chains. For $n \in \mathcal{N}$ with $2n - 1 \leq M$ we add the implied constraints $C_n(2n - 1 - i, i) \neq C_\infty(2n - 1 - i, i)$ of Lemma 2.3 (their derivation only uses (2) at the cell $(2n - 2, 1)$, which is part of the encoding).

Pumping. For all $4 \leq n < n' \leq N'$ (with $2N' - 2 \leq M$) we add one clause stating that J_n and $J_{n'}$ differ on $\text{cone}_n(n - 1, 0)$, introducing an auxiliary “difference” variable for every pair of non-constant input cells (Lemma 3.3).

Determinacy links. For $n < n'$ in $\mathcal{N}$ and $\kappa_0 \geq 0$ an auxiliary variable $E_{n,n'}(\kappa_0)$ is made equivalent to “ J_n and $J_{n'}$ coincide on the positions $0, \dots, \kappa_0$ ”. For every relative cell (τ, κ) of R_n whose dependency cone is right-anchored and contained in the positions $0, \dots, \kappa_0$ we add $E_{n,n'}(\kappa_0) \Rightarrow (C_n(n - 1 + \tau, n - \kappa) = C_{n'}(n' - 1 + \tau, n' - \kappa))$, and, for the firing cells $\tau = n - 1$, the clause $\neg E_{n,n'}(\kappa_0)$ (Lemmas 3.1 and 3.3). These clauses are implied by the others but allow the solver to transfer information between lines.

Symmetry breaking. Permuting the auxiliary states maps solutions to solutions. We require that, in the time-major enumeration of the encoded half-line cells, A_{j+1} occurs only after a first occurrence of A_j ; every solution can be renamed to satisfy this. For $k = 5$ this halves the search space.

Soundness checks. Besides the proofs above, every constraint family was tested against an independent ground truth: Mazoyer’s six-state solution, extracted mechanically from Duprat’s Coq formalization [16], satisfies the formula $MT_6(\mathcal{N}; 78, 40)$ for $\mathcal{N} = \{3, \dots, 12\}$ with return chains, pumping and links (the formalization does not cover $n = 2$), and it satisfies $MT_6(\{3, \dots, 14\}; 26)$ with symmetry breaking under exactly one of the $3!$ renamings of its auxiliary states, as it must. Conversely, every model found by the solvers was decoded and re-simulated by an independent brute-force checker (`fsspcheck.c`), which confirmed minimal-time synchronization for all encoded lengths.

5. Four states: a certified proof

All formulas are produced by the generator `gencnf.py`; models are decoded into rule tables and re-simulated by the independent brute-force checker `fsspcheck.c`. The programs were written with the help of a generative AI system (see the declaration before the references). Solvers: Kissat 4.0.4 [23] and CaDiCaL 3.0.1 [24] (or CaDiCaL 1.9.5 through PySAT [25] for incremental runs). Times are single-core wall-clock times on a cloud virtual CPU.

Theorem 5.1. *No rule with four states $\{L, G, A, F\}$ satisfying (1) synchronizes all lines of lengths $2, \dots, 9$ in minimal time. In particular there is no four-state minimal-time solution, even without the condition $\delta(*, L, L) = L$.*

Proof. By Lemmas 2.2 and 2.3 and (2) every such rule yields a model of $MT_4(\{2, \dots, 9\}; 16)$ (for $k = 4$ there is a single auxiliary state and no symmetry breaking). CaDiCaL proves this formula unsatisfiable in 51 s and emits an LRAT proof [26] of 177 MB, which `lrat-check` of the `drat-trim` suite [27] verifies in 5.7 s. The SHA-256 digests of the formula and of the proof are listed in Appendix D. $\square$

The bound 9 is sharp for the plain formula: $MT_4(\{2, \dots, 8\}; 14)$ is satisfiable, and an independent depth-first enumeration (program `fsspdfs.c`, which fills the table lazily in the order in which transitions are needed) finds the numbers of partial rules listed in Table 3, counted on the transitions they

largest length N	5	6	7	8	9
partial rules synchronizing $2, \dots, N$	828,670	64,299	536	27	0

Table 3: Four states: number of partial rules (restricted to the transitions they use) that synchronize every line of length $2, \dots, N$ in minimal time, by exhaustive enumeration ($1.09 \cdot 10^8$ search nodes).

use; it reaches zero exactly at the lengths $2, \dots, 9$, in agreement with the solver. With the half-line extended to anti-diagonal 78 the lengths $2, \dots, 8$ already suffice. The pumping constraints alone are weak for four states: together with the half-line up to anti-diagonal 78, the lengths $2, \dots, 7$ remain satisfiable.

6. Five states: the frontier

6.1. Lucky rules

For five states the table has 94 free transitions with five values each, against 43 with four values for four states (in both cases the neighbourhoods $(*, y, *)$ and the two quiescence transitions are excluded): the search space grows by a factor of about $5^{94}/4^{43} \approx 10^{40}$. Table 4 lists representative instances. All of them are satisfiable, and in each case the decoded rule fails for some length that was not encoded; we call such rules *lucky*. Two kinds occur. Rules of the first kind have a *periodic* half-line behind the front (Figure 8(b)); they can even synchronize an isolated long line such as $n = 20$, but they violate the pumping lemma massively (155 of the pairs $4 \leq n < n' \leq 40$ for the rule shown). Rules of the second kind have a chaotic half-line which, without the half-line constraints, typically fires at some point. Once the half-line is extended and the pumping constraints are imposed, the half-lines produced by the solver change character: in the models we examined in detail—Figure 8(c) and the rules of Theorem 6.1, whose common half-line consists of a front of G’s, a solid wake of G’s and a region $ALAL \dots$ behind a boundary of speed exactly $1/3$ —a region delimited by a line of speed $1/3$ appears, as predicted by Theorem 3.4. Chaotic half-lines that never fire satisfy the pumping constraints as well; one of them is discussed after Corollary 6.2.

6.2. Five-state rules for all lengths up to fourteen

Theorem 6.1. *The five-state rules δ_{12} and δ_{13} of Tables C.8 and C.9 have the following properties.*

instance ($k = 5$)	result	time
$\text{MT}_5(\{2, \dots, 10\}; 18)$	SAT, fails at $n = 11$	10.8 s
$\text{MT}_5(\{2, \dots, 12\}; 22)$	SAT, fails at $n = 13$	≈ 20 min
$\text{MT}_5(\{2, \dots, 9, 16\}; 30)$	SAT, fails at $n = 10$	23.6 s
$\text{MT}_5(\{2, \dots, 9, 20\}; 38)$	SAT, fails at $n = 10$	82.4 s
$\text{MT}_5(\{2, \dots, 8\}; 78, 40)$	SAT, fails at $n = 9$	68.7 s
$\text{MT}_5(\{2, \dots, 10\}; 78, 40)$	SAT, fails at $n = 11$	755.6 s
$\text{MT}_5(\{2, \dots, 12\}; 78, 40) + \text{links} + \text{uniform pre-firing } A^n$	SAT, fails at $n = 13$	1930 s

Table 4: Five-state instances. $\text{MT}_k(\mathcal{N}; M, N')$: lengths $\mathcal{N}$, half-line up to anti-diagonal M , pumping pairs up to N' (none if N' is omitted).

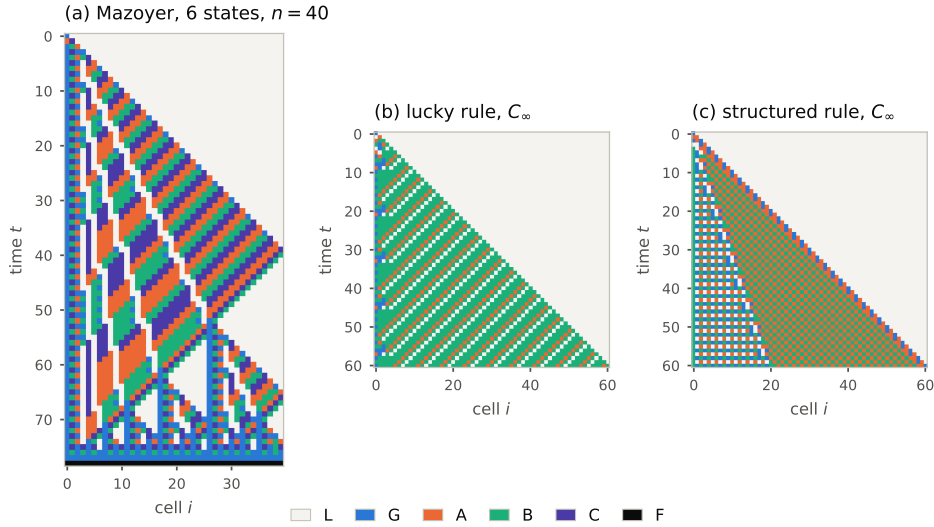


Figure 8: Space-time diagrams, time downwards. (a) Mazoyer’s six-state solution on $n = 40$ cells; the last row is the firing row. (b) Half-line of a five-state rule that synchronizes the lengths $2, \dots, 9$ and 20 but violates the pumping lemma: its background is periodic. (c) Half-line of a five-state rule found under the pumping constraints: the boundary between the checkered region and the diagonal background has speed $1/3$.

- (a) δ_{12} synchronizes the line of length n at time $2n - 2$ for every $2 \leq n \leq 12$, and one step before firing all cells are in state A. It does not synchronize the line of length 13: a cell fires at time 23.
- (b) δ_{13} synchronizes the line of length n at time $2n - 2$ for every $2 \leq n \leq 13$, and one step before firing all cells are in state A whenever $4 \leq n \leq 13$.

It does not synchronize the line of length 14: a cell fires at time 21.

(c) *The half-line diagrams of δ_{12} and δ_{13} coincide on all cells (t, i) with $t + i \leq 78$; these cells use 22 neighbourhoods besides (L, L, L) , on which the two rules agree, and none of them is a firing cell.*

(d) *Both rules satisfy $\text{PUMP}(n, n')$ for all $4 \leq n < n' \leq 40$.*

Proof. Each property is a finite computation on the explicit tables, carried out by direct simulation: (a), (b) and the failures at $n = 13$ and $n = 14$ with the checker `fsspcheck.c`, (c) with `check_halfline.py` and `germcompare.py`, and (d) with `pumping.py`, which recomputes the cones of Lemma 3.2 by breadth-first search (Appendix D). $\square$

The rule δ_{12} was found by Kissat on $\text{MT}_5(\{2, \dots, 12\}; 78, 40)$ with links and the uniform pre-firing constraint (Table 4); δ_{13} was found by the same solver after fixing the 22 half-line neighbourhoods of δ_{12} and adding the length 13, in 108 s. The two rules differ in 57 of the remaining entries. Consequently, the necessary conditions used in this paper, restricted to lengths at most 13 (even together with the pumping constraints up to 40 and the half-line up to anti-diagonal 78), are satisfiable (Proposition 6.3 below extends this to length 14), and the requirement “uniform configuration before firing”, satisfied by Mazoyer’s solution, does not help for the lengths up to 12. The half-line shared by the two rules consists of a front of G’s, a solid wake of G’s and, behind a boundary of speed exactly $1/3$, a region $ALAL\dots$ that is periodic in time and space. By the left-border barrier this architecture cannot be completed to a solution:

Corollary 6.2. *No rule that agrees with δ_{12} on the 22 neighbourhoods of Theorem 6.1(c) synchronizes the line of length 518 in minimal time. In particular no minimal-time solution has the half-line of δ_{12} and δ_{13} below anti-diagonal 78.*

Proof. Simulating the half-line with these 22 neighbourhoods (and (L, L, L)) only, no other neighbourhood occurs on the cells with $t + i \leq 1034$; hence every such rule has the same diagram C_∞ there. On the anti-diagonal $t + i = 1034$ the cells $2, \dots, 259$ are all in state A, and on $t + i = 1033$ the cells $2, \dots, 258$ are all in state L. Lemma 3.5 with $k = 5$, $n = 518$, $s = 2$, $L = 1$ and $Y = 259$ would give $257 \leq 4^4 = 256$. $\square$

So no choice of the remaining transitions can rescue this architecture. The lemma only bites at length 518, whereas the two rules themselves already

fail at lengths 13 and 14. The regularity certificate of Lemma 3.9 (with $P = 6$) gives a second proof of the last statement of Corollary 6.2 that does not use a particular length.

The frontier is not tied to this architecture, and it is not at thirteen. The classification of Section 6.3 below left a few germs of complexity 16 undecided. For one of them, Kissat completes the germ to a rule for the lengths $2, \dots, 13$ in 442 s (file `delta13_chaotic.txt`); fixing the same germ and asking for the lengths $2, \dots, 14$, together with the links and the pumping constraints of Section 4, it finds in 1758 s a rule for all lengths up to fourteen, with a half-line of an entirely different kind (Figure 10).

Proposition 6.3. *The five-state rule δ_{14} of Table C.10 synchronizes the line of length n at time $2n - 2$ for every $2 \leq n \leq 14$ and fails for $n = 15$ (a cell fires at time 22). Its half-line diagram uses 16 neighbourhoods besides (L, L, L) on all cells with $t + i \leq 10^4$, has no firing cell there, and satisfies $\text{PUMP}(n, n')$ for all $4 \leq n < n' \leq 800$; for $n \leq 5000$, $s \leq 3$ and $L \leq 24$ its input anti-diagonals have no stretch to which Lemma 3.5 applies.*

Proof. Direct simulation with the programs `fsspcheck.c`, `germcompare.py`, `germext.c` and `pumping.py`. $\square$

The half-line of δ_{14} (Figure 10(b)) is chaotic. For $3 \leq t < 1500$, cell 1 takes only the values L, G, cell 2 only L, A, and all other cells only L, B; on the latter the germ is the elementary cellular automaton *rule 22* in the numbering of [28] (a cell becomes B if and only if exactly one cell of its neighbourhood is B), a classical example of chaotic behaviour, driven at its left end by the two border columns. The reflected triangles of δ_{14} end with a recognisable wave: for $4 \leq n \leq 14$ the configuration one step before firing is $L G^{n-2} B$, and the state G, which the half-line uses only in its first column, marks the firing wave against the chaotic background. This mechanism does not extend: with the germ of δ_{14} fixed, no rule synchronizes all lengths $2, \dots, 15$ with these configurations before firing (Kissat, unsatisfiable after about 30 minutes); without this restriction the lengths $2, \dots, 15$ remain undecided after several hours. The half-line has non-periodic structure everywhere, and in the ranges of Proposition 6.3 neither barrier of Section 3 excludes it; the only arguments that we can apply to it are completion tests for longer lines, which are expensive. Consequently, the necessary conditions used in this paper, restricted to lengths at most 14 (together with the pumping constraints up to 40 and the half-line up to anti-diagonal 78), are satisfiable, and a refutation of five-state minimal-time solutions along these lines must involve lines of length at least 15.

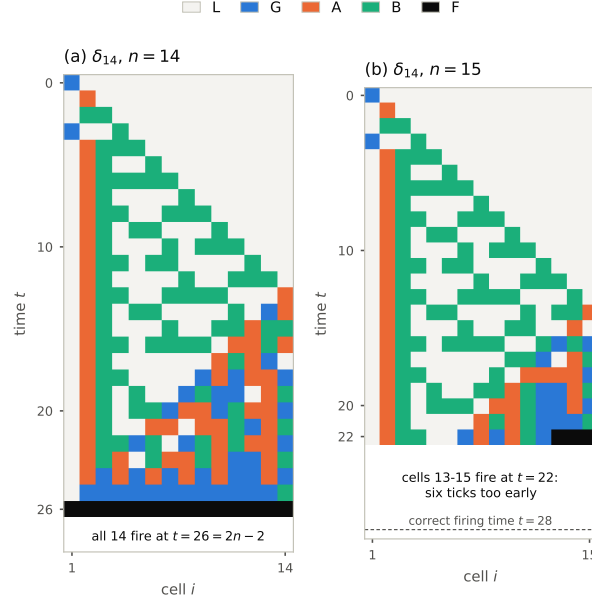


Figure 9: The rule δ_{14} on the lines of lengths 14 and 15 (time runs downwards, one square per cell and time step, coloured by state). (a) All 14 cells fire at $t = 26 = 2n - 2$. (b) The cells 13, 14 and 15 fire at $t = 22$, before the correct firing time $t = 28$.

Figure 9 shows δ_{14} at work on the lines of lengths 14 and 15. Four further rules synchronize every line of length at most 14, on other germs (Appendix B.1): δ_{14}^b , whose half-line is eventually regular, so that no completion of its germ is a solution (Lemma 3.9); δ_{14}^c and δ_{14}^d , on the two surviving germs of complexity 15, which share one half-line up to a renaming of the states; and δ_{14}^e , on a germ of complexity 17. All five fail at $n = 15$ in the same way: cells in the right half of the line fire one to seven steps early (Table B.7).

6.3. Half-lines of low complexity

For a rule δ and a bound M , the *germ* of δ below M is the restriction of δ to the neighbourhoods $(C_\infty(t-1, i-1), C_\infty(t-1, i), C_\infty(t-1, i+1))$ of the cells (t, i) with $t \geq 1$ and $t+i \leq M$ of its half-line diagram, and its *complexity* $c_M(\delta)$ is the number of these neighbourhoods other than (L, L, L) . The germ determines C_∞ on all cells with $t+i \leq M$. The rules of Theorem 6.1 have $c_{78} = 22$ and Mazoyer's rule has $c_{78} = 57$.

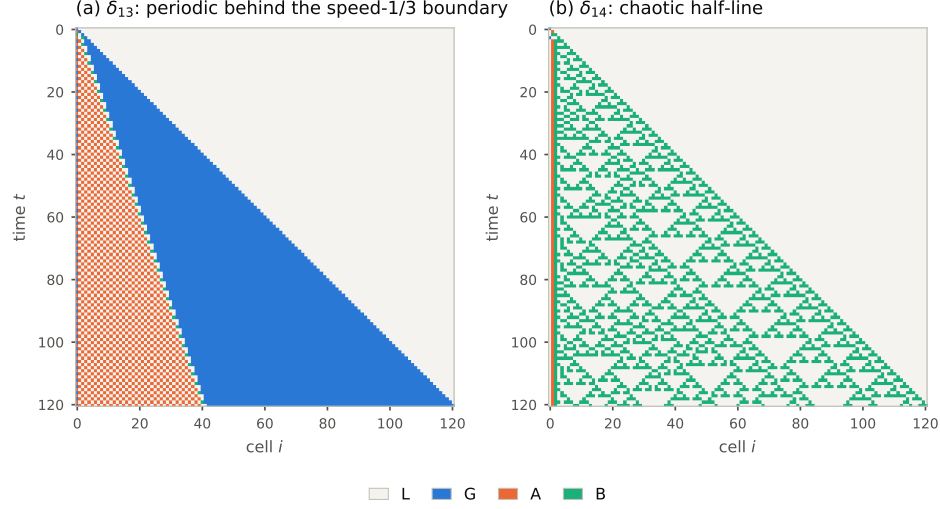


Figure 10: Half-lines of the two frontier architectures (time downwards, $0 \leq t \leq 120$). (a) The common half-line of δ_{12} and δ_{13} : a front and a solid wake of G's and a checkered region of A's and L's behind a boundary of speed $1/3$; the checkered region is excluded by Lemma 3.5 (Corollary 6.2). (b) The half-line of δ_{14} , determined by 16 neighbourhoods; its irregular pattern escapes both barriers.

Proposition 6.4. *Every five-state minimal-time solution δ satisfies $c_{78}(\delta) \geq 15$.*

Proof (computer-assisted). Let δ be a five-state minimal-time solution with $c_{78}(\delta) \leq 14$. Order the cells of C_∞ by anti-diagonal and, within an anti-diagonal, by increasing time; after exchanging the names A and B if necessary, A occurs before B in this order (if at all). By (2), Lemmas 2.3 and 3.3, the germ of δ below 78 produces no firing cell, never produces L at the front, and satisfies $\text{PUMP}(n, n')$ for $4 \leq n < n' \leq 40$. The program `germdfs.c` enumerates all germs with these properties, with at most 14 neighbourhoods besides (L, L, L) and with A before B, by a depth-first search over the cells in the above order in which the value of a neighbourhood is chosen when it first occurs: $1.0 \cdot 10^8$ nodes, 244 germs. For 230 of them the formula $\text{MT}_5(\{2, \dots, 10\}; 78, 39)$ (without symmetry breaking, which the enumeration has already performed) together with the germ is unsatisfiable; a single formula in which a selector variable chooses the germ is refuted by CaDiCaL in 4s, with an LRAT proof (11 MB) checked by `lrat-check` in under a second. Each of the remaining 14 germs determines C_∞ far beyond anti-diagonal 78 (no new neighbourhood occurs up to anti-diagonal 3300),

complexity bound $c_{78} \leq$	14	16
search nodes of <code>germdfs.c</code>	$1.0 \cdot 10^8$	$1.5 \cdot 10^9$
germs (non-firing, front $\neq L$, PUMP to 40)	244	9,787
refuted by the completion test for $2, \dots, 10$ (LRAT)	230	8,824
then refuted by Lemma 3.5 or Lemma 3.3	14	931
then refuted by the completion test for $2, \dots, 13$	–	8
then refuted by Lemma 3.9 (eventually regular)	–	8
remaining (not certified regular)	0	16
refuted by Lemma 3.5 or Lemma 3.3 alone	218	$\geq 8,263$

Table 5: Classification of the half-line germs of low complexity (Section 6.3). The last row applies the half-line arguments to all germs, without completion tests (half-lines simulated up to anti-diagonal 3300 for $c_{78} \leq 14$ and 1300 for $c_{78} \leq 16$).

and on these extended diagrams Lemma 3.5 applies with $s = 1$: for each germ there are $n \leq 163$ and $L \leq 5$ such that the two input anti-diagonals of the line of length n are L -periodic on $[1, Y]$ with $Y - 1 > 16L$ (for instance $n = 28$, $L = 1$, $Y = 19$). Hence no rule containing one of the 244 germs is a minimal-time solution. $\square$

The same pipeline, run on the 9,787 germs of complexity at most 16, shows how the two kinds of arguments share the work (Table 5, details in Appendix B.2). The completion test for the lengths $2, \dots, 10$ refutes 8,824 germs, with one LRAT certificate; the lemmas, applied to half-lines extended up to anti-diagonal 10^4 , refute 931 of the survivors; a completion test for the lengths $2, \dots, 13$ and Lemma 3.9 refute 8 more each. Used alone, the half-line arguments refute at least 8,263 of the 9,787 germs, without any SAT call. The 16 germs that remain, among them the germ of δ_{14} , are irregular as far as our certificates can tell, and those we inspected are chaotic. Two of them have complexity exactly 15 and share one half-line up to a renaming of the states; both extend to rules for all lengths up to 14 (δ_{14}^c and δ_{14}^d). Raising the bound of Proposition 6.4 to 16 therefore amounts to refuting this half-line, which requires lines of length at least 15; a cube-and-conquer attack on the lengths $2, \dots, 15$ refuted thousands of cubes but left the question undecided (Appendix B.4). For the 43,294 germs of complexity exactly 17 the same filters leave 187, again with an LRAT certificate for the completion test, and one of the survivors yields δ_{14}^e (Appendix B.3).

6.4. Where the difficulty lies

Once a germ is fixed, the half-line is determined up to anti-diagonal 78 and only the reflected triangles remain. For three lucky rules of Table 4, those for $\{2, \dots, 9, 16\}$, for $\{2, \dots, 9, 20\}$ and, found with the half-line up to anti-diagonal 78, for $2, \dots, 10$, Kissat decides the completion problem of the germ in less than one second: the first two germs have no completion even for the lengths $2, \dots, 8$, the third has completions for the lengths up to 10 but not up to 11. (The half-line of the rule for $2, \dots, 12$, found with the half-line only up to anti-diagonal 22, fires below anti-diagonal 78.) Fixing the 58 transitions that Mazoyer’s rule uses on its half-line up to anti-diagonal 78 leaves a completion problem for the lengths $3, \dots, 12$ (without symmetry breaking) that Kissat solves in 4 s. The chaotic germs are the exception: the completions of the germ of δ_{14} for the lengths up to 12, 13 and 14 take 2 s, 442 s and 1758 s (the last one with links), and for the lengths up to 15 a Kissat run without links stopped without an answer after four hours.

To measure the size of the whole search, we estimated the natural look-ahead search tree—half-line transitions fixed in order of first use, anti-diagonal by anti-diagonal, a node being discarded when the full formula with the partial table as assumptions is refuted within 3000 conflicts—with Knuth’s random-probe estimator [29]: a probe follows a random path from the root to a leaf, and if $d_1, \dots, d_D$ are the numbers of children of the nodes of the path before the leaf, then $1 + d_1 + d_1 d_2 + \dots + d_1 \dots d_D$ is an unbiased estimate of the number of nodes. For four states ($\text{MT}_4(\{2, \dots, 9\}; 16, 8)$ with links) the mean of this estimate over 30 probes is 25.5 nodes. For five states ($\text{MT}_5(\{2, \dots, 12\}; 78, 39)$ with links) three probes reach depths 30 to 36; their last terms $d_1 \dots d_D$, which are lower bounds for their estimates, are $4.2 \cdot 10^{11}$, $3.1 \cdot 10^{17}$ and $8.3 \cdot 10^{18}$, and the mean estimate is $6.7 \cdot 10^{18}$ nodes. Adding the lengths up to 18 changes little (last terms $2.1 \cdot 10^{11}$, $5.1 \cdot 10^{16}$ and $4.2 \cdot 10^{18}$, mean estimate $2.2 \cdot 10^{18}$). The estimator has a large variance and three probes are few, but every five-state probe exceeds the four-state estimate by at least nine orders of magnitude, in line with the fact that our runs on the unrestricted five-state formulas with the pumping constraints for the lengths $2, \dots, N$ with $N = 12, 14$ and 18 (up to about 1.5 CPU-hours each) did not terminate. Clause learning compresses such trees considerably, and cube-and-conquer [30] parallelizes them well, but a complete five-state search along these lines is a computation of a much larger scale than the ones reported here.

7. Discussion

What the pumping lemma buys. Lemma 3.3 turns an infinite family of requirements on the reflected triangles into constraints on the half-line diagram only. It is satisfied by Mazoyer’s solution, as it must be, and it eliminates, at negligible cost, the whole class of rules whose half-line becomes periodic behind the front too early. Theorem 3.4 shows that the constraint has a transparent geometric meaning: in every minimal-time solution the non-periodic part of the half-line reaches the line $i = t/3$. For Mazoyer’s solution every depth-row $j < 700$ is 3-periodic from time $2j + 1$ or $2j + 2$, i.e. the periodic zone behind the front is bounded by a signal of speed $1/2$, comfortably within the bound.

Why lucky rules are hard to kill. Small sets of necessary conditions are typically met by rules that are not solutions; the question is how many lengths are needed before all such rules disappear (if no solution exists, some finite set of lengths suffices, since there are finitely many rules). For four states the answer is 9 (or 8 with a long half-line). For five states the rule space is larger by roughly 40 orders of magnitude (5^{94} versus 4^{43} transition tables), and our experiments show rules for all lengths up to 14, and partial rules for any single additional length we tried. Under the pumping constraints the surviving rules are of two kinds. Either their half-lines consist of finitely many periodic regions separated by signals, with a boundary of speed $1/3$ (as for δ_{12} and δ_{13}); such half-lines are periodic in a wedge at the left border and are excluded by Theorem 3.6, although only for long lines. Or their half-lines are chaotic (as for δ_{14}); these pass both barriers in the ranges we checked, and at present only completion tests on longer lines can exclude them.

The mirrored construction. The cone of Lemma 3.2 ends where the anti-diagonal $t + i = 2n - 2$ meets the line $i = t/3$. By Lemma 3.1, the part of R_n whose cone lies in the periodic zone behind the front is the same for all large n of a given residue class, and the information that makes the right end fire exactly at time $2n - 2$ enters R_n near that crossing and travels to the right end at maximal speed. Heuristically, the right half of every reflected triangle therefore has to behave like a *mirrored* minimal-time synchronization, started by the reflection and running on the background left behind the front instead of on quiescent cells. A five-state solution would have to realize both constructions—the direct one on the quiescent background and the mirrored one on the front’s background—with the same four working states. We regard this tension as the structural reason why five states are hard.

Remark 3.7 turns one half of the heuristic into a statement: seen from the right end, the reflected triangles cannot be periodic over long stretches, so the mirrored construction needs its own non-periodic structure near the right border. The five rules for all lengths up to 14 (Section 6 and Appendix B.1) fail at $n = 15$ in the same way, which is consistent with this picture: in each of them the first cells to fire are in the right half of the line (cells 9 to 15), at times between 21 and 27 instead of 28, and no cell near the left border fires early. A mirrored version of Theorem 3.4 is still missing (the mirrored problem has no border at its far end), and turning the tension into an impossibility proof is the most promising route to a conceptual answer.

The left half. The same time budget applies on the left. The reflected wave reaches cell i at time $2n - 1 - i$ and every cell fires at time $2n - 2$; information created behind the wave cannot overtake it, so the cells to the left of the midpoint can only combine what the half-line has stored around them with what a bounded number of chains immediately behind the wave bring from the right. Theorem 3.6 turns this into a theorem: the half-line cannot be periodic in a wedge at the left border. In the classical constructions the required structure is the hierarchy of slow signals whose crossings with the wave mark the recursive division points; the common half-line of the rules of Theorem 6.1 has none, and Corollary 6.2 excludes it. The bound of Lemma 3.5 is exponential in s and linear in the period, so the lemma is weak for short lines; a sharper version, or a version for half-lines that are only *eventually* periodic along the anti-diagonals, would strengthen the search of Section 6 considerably.

Chaotic half-lines. Half-lines that pass Theorems 3.4 and 3.6 can be hierarchical, like the half-line of Mazoyer’s six-state solution, or chaotic. All five-state half-lines that passed every test of Section 6.3 and that we inspected are chaotic, like the one of δ_{14} , which is essentially the elementary rule 22. A germ of complexity c leaves $94 - c$ free transitions, for the germ of δ_{14} that is 78, i.e. $5^{78} \approx 2^{181}$ completions, and each additional length imposes of the order of n independent-looking conditions on the reflected triangle. If these conditions behaved like independent events of bounded probability, the completions of a chaotic germ would run out around $n \approx 20$; the rule δ_{14} and the completion runs of Section 6.3 are consistent with this heuristic, which we have not been able to turn into a proof. For the left border the difficulty is concrete: by the proof of Lemma 3.5, the firing of the cell $2s$ depends only on the input anti-diagonals at the cells $i \geq s$ and on a band of $2s$ chains; in a chaotic medium without markers near the border, such a

band has no evident way of recognising the cell $2s$.

Open problems.

- (1) Does a five-state minimal-time solution exist? By Proposition 6.3, a negative answer along the lines of Theorem 5.1 requires lines of length at least 15.
- (2) Is there a necessary condition, in the spirit of Lemma 3.3, that constrains the reflected triangles themselves and not only the half-line?
- (3) What is the smallest N such that some five-state rule is $\{2, \dots, N\}$ -minimal but no five-state rule is $\{2, \dots, N + 1\}$ -minimal, if such an N exists? We know $N \geq 14$.
- (4) Can the bound $(k - 1)^{2s}L$ of Lemma 3.5 be replaced by one that is polynomial in s , or by one that also covers anti-diagonals that are periodic only up to a bounded number of defects? Either would make the lemma effective for the self-similar half-lines that survive the filters of Section 6.3.

CRedit authorship contribution statement

Kia Keng Giam: Conceptualization, Methodology, Formal analysis, Investigation, Validation, Writing – review & editing.

Beichen Sun: Conceptualization, Methodology, Formal analysis, Investigation, Validation, Writing – review & editing.

Yaoqi Zhao: Conceptualization, Methodology, Formal analysis, Investigation, Validation, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data availability

All programs, formula generators, rule tables, germ lists and experiment logs are publicly available in the repository <https://github.com/giamkiakeng/proj1>, directory `fssp5`. The LRAT certificates are not stored (the one of Theorem 5.1 has 177MB); they are regenerated deterministically by the commands of Appendix D and of the file `results/germs/CERTIFICATES.txt`, which list their SHA-256 digests.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Claude (Anthropic), a generative AI system, in order to develop and check the mathematical arguments, to design, write and run the programs, to carry out the computer experiments, to prepare the figures and to draft and proofread the manuscript. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

References

- [1] E. F. Moore, The firing squad synchronization problem, in: E. F. Moore (Ed.), *Sequential Machines: Selected Papers*, Addison-Wesley, Reading, MA, 1964, pp. 213–214.
- [2] E. Goto, A minimal time solution of the firing squad problem, Dittoed course notes for Applied Mathematics 298, Harvard University, pp. 52–59 (1962).
- [3] A. Waksman, An optimum solution to the firing squad synchronization problem, *Information and Control* 9 (1) (1966) 66–78.
- [4] J. Mazoyer, An overview of the firing squad synchronization problem, in: C. Choffrut (Ed.), *Automata Networks*, Vol. 316 of *Lecture Notes in Computer Science*, Springer, 1988, pp. 82–94.
- [5] H. Umeo, Firing squad synchronization problem in cellular automata, in: R. A. Meyers (Ed.), *Encyclopedia of Complexity and Systems Science*, Springer, 2009, pp. 3537–3574.
- [6] R. Balzer, An 8-state minimal time solution to the firing squad synchronization problem, *Information and Control* 10 (1) (1967) 22–42.

- [7] H.-D. Gerken, Über Synchronisationsprobleme bei Zellularautomaten, Diplomarbeit, Institut für Theoretische Informatik, Technische Universität Braunschweig (1987).
- [8] J. Mazoyer, A six-state minimal time solution to the firing squad synchronization problem, *Theoretical Computer Science* 50 (2) (1987) 183–238.
- [9] P. Sanders, Massively parallel search for transition-tables of polyautomata, in: *Parcella '94: Proceedings of the VI International Workshop on Parallel Processing by Cellular Automata and Arrays*, Akademie Verlag, Berlin, 1994, pp. 99–108.
- [10] T. Correa, B. Gustavo, L. Lemos, A. Settle, An overview of recent solutions to and lower bounds for the firing synchronization problem, *arXiv:1701.01045* (2017).
- [11] M. Clergue, S. Verel, E. Formenti, An iterated local search to find many solutions of the 6-states firing squad synchronization problem, *Applied Soft Computing* 66 (2018) 449–461.
- [12] T. T. Nguyen, L. Maignan, Exploring millions of 6-state FSSP solutions: the formal notion of local CA simulation, in: *Cellular Automata and Discrete Complex Systems (AUTOMATA 2020)*, Vol. 12286 of *Lecture Notes in Computer Science*, Springer, 2020, pp. 1–13.
- [13] H. Umeo, T. Yanagihara, A smallest five-state solution to the firing squad synchronization problem, in: *Machines, Computations, and Universality (MCU 2007)*, Vol. 4664 of *Lecture Notes in Computer Science*, Springer, 2007, pp. 291–302.
- [14] J.-B. Yunès, A 4-states algebraic solution to linear cellular automata synchronization, *Information Processing Letters* 107 (2) (2008) 71–75.
- [15] H. Umeo, N. Kamikawa, J.-B. Yunès, A family of smallest symmetrical four-state firing squad synchronization protocols for ring arrays, *Parallel Processing Letters* 19 (2) (2009) 299–313.
- [16] J. Duprat, Firing Squad Synchronization Problem, Coq user contribution `FiringSquad` (formal proof of the correctness of Mazoyer’s six-state solution), LIP, ENS Lyon; archived at <https://github.com/rocq-archive/firing-squad> (1997).
- [17] M. L. Minsky, *Computation: Finite and Infinite Machines*, Prentice-Hall, Englewood Cliffs, NJ, 1967.

- [18] J. Mazoyer, On optimal solutions to the firing squad synchronization problem, *Theoretical Computer Science* 168 (2) (1996) 367–404.
- [19] A. Settle, J. Simon, Smaller solutions for the firing squad, *Theoretical Computer Science* 276 (1–2) (2002) 83–109.
- [20] A. Berthiaume, T. Bittner, L. Perković, A. Settle, J. Simon, Bounding the firing synchronization problem on a ring, *Theoretical Computer Science* 320 (2–3) (2004) 213–228.
- [21] M. J. H. Heule, O. Kullmann, V. W. Marek, Solving and verifying the Boolean Pythagorean triples problem via cube-and-conquer, in: *Theory and Applications of Satisfiability Testing (SAT 2016)*, Vol. 9710 of *Lecture Notes in Computer Science*, Springer, 2016, pp. 228–245.
- [22] M. J. H. Heule, Schur number five, in: *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence (AAAI-18)*, 2018, pp. 6598–6606.
- [23] A. Biere, K. Fazekas, M. Fleury, M. Heisinger, CaDiCaL, Kissat, Para-cooba, Plingeling and Treengeling entering the SAT Competition 2020, in: *Proceedings of SAT Competition 2020: Solver and Benchmark Descriptions*, Vol. B-2020-1 of *Department of Computer Science Report Series B*, University of Helsinki, 2020, pp. 51–53.
- [24] A. Biere, T. Faller, K. Fazekas, M. Fleury, N. Froleyks, F. Pollitt, CaDiCaL 2.0, in: *Computer Aided Verification (CAV 2024)*, Vol. 14681 of *Lecture Notes in Computer Science*, Springer, 2024, pp. 133–152.
- [25] A. Ignatiev, A. Morgado, J. Marques-Silva, PySAT: a Python toolkit for prototyping with SAT oracles, in: *Theory and Applications of Satisfiability Testing (SAT 2018)*, Vol. 10929 of *Lecture Notes in Computer Science*, Springer, 2018, pp. 428–437.
- [26] L. Cruz-Filipe, M. J. H. Heule, W. A. Hunt, Jr., M. Kaufmann, P. Schneider-Kamp, Efficient certified RAT verification, in: *Automated Deduction – CADE 26*, Vol. 10395 of *Lecture Notes in Computer Science*, Springer, 2017, pp. 220–236.
- [27] N. Wetzler, M. J. H. Heule, J. Hunt, Warren A., DRAT-trim: efficient checking and trimming using expressive clausal proofs, in: *Theory and Applications of Satisfiability Testing (SAT 2014)*, Vol. 8561 of *Lecture Notes in Computer Science*, Springer, 2014, pp. 422–429.

- [28] S. Wolfram, Statistical mechanics of cellular automata, *Reviews of Modern Physics* 55 (3) (1983) 601–644. doi:10.1103/RevModPhys.55.601.
- [29] D. E. Knuth, Estimating the efficiency of backtrack programs, *Mathematics of Computation* 29 (129) (1975) 121–136.
- [30] M. J. H. Heule, O. Kullmann, S. Wieringa, A. Biere, Cube and conquer: guiding CDCL SAT solvers by lookaheads, in: *Hardware and Software: Verification and Testing (HVC 2011)*, Vol. 7261 of *Lecture Notes in Computer Science*, Springer, 2012, pp. 50–65.

Appendix A. Glossary of terms and symbols

This appendix defines every term and every symbol used in the paper; the most frequent notation is summarised on the page *Notation at a glance* before the introduction. The entries are grouped by topic and ordered so that each definition uses only notions defined before it; a reference in parentheses points to the place where a notion is used or treated formally. A few words are used only *descriptively*, to name what a picture shows (for instance *chaotic*); they are collected in one entry of Appendix A.3, carry no formal content, and no statement of the paper depends on them. The symbols are summarised in Table A.6 at the end of the appendix.

Appendix A.1. General conventions

Integers. $\mathbb{N} = \{0, 1, 2, \dots\}$. For a real number x , $\lfloor x \rfloor$ and $\lceil x \rceil$ are the largest integer $\leq x$ and the smallest integer $\geq x$. For integers, $L \mid h$ means that L divides h , $a \equiv b \pmod{L}$ means $L \mid a - b$, and $y \bmod L$ is the remainder of y in $\{0, \dots, L - 1\}$. The limit superior of a sequence is written $\limsup$. “For n large” means “for every n from some value on”.

Sets and maps. $|X|$ is the number of elements of a finite set X , $X \times Y$ the Cartesian product and $X \setminus Y$ the difference. A *partial map* is a map defined on a subset of its nominal domain.

Words. A *word* over an alphabet X is a finite sequence of elements (*letters*) of X , written without separators; $|w|$ is the *length* of the word w , words are multiplied by concatenation, and w^a is the concatenation of $a \geq 0$ copies of w . A word written as a map $w : I \rightarrow X$ on an interval I of integers is *L-periodic*, for an integer $L \geq 1$ called a *period*, if $w(y + L) = w(y)$ whenever y and $y + L$ belong to I ; a *periodic stretch* of a word is an interval on which it is *L-periodic* for a given L . A *defect* of w with respect to L is a position y with $w(y + L) \neq w(y)$; w is *L-periodic up to d defects* if it has at most d of them.

Eventually periodic. A sequence $(x_t)_{t \geq 0}$ is *eventually periodic* if there are $\theta \geq 0$ and $P \geq 1$ such that $x_{t+P} = x_t$ for all $t \geq \theta$; P is a period, the least such θ is the *transient*, the terms before it form the *transient part* and the terms from θ on the *periodic regime*.

Typography. Sans-serif capitals L, G, A, B, F always denote states; the italic L is always a period (Lemma 3.5); the italic T is a time, and $T_{e,d}$ is a propositional variable (Appendix A.6). Program names in typewriter font (`fsspcheck.c`, ...) refer to files of the accompanying repository, listed in Appendix D.

Appendix A.2. The firing squad

Cellular automaton (CA). A row of identical finite automata, the *cells*, that change their states simultaneously at every tick of a discrete clock, each as a function of its own state and of the states of its two nearest neighbours. Here the row is either finite (a *line*) or infinite to the right (the *half-line*, Appendix A.3).

Cell; position. The cells of a line are numbered $1, \dots, n$ from left to right; the number i of a cell is its *position*. In the figures a cell is drawn as a soldier.

Time; step. Time is discrete, $t \in \mathbb{N}$; one (*time*) *step* updates all cells at once.

State; state set S ; number of states k . At every time each cell is in one *state* of a finite set S , and $k := |S|$. A rule or solution is *k-state* if $|S| = k$. The *state complexity* of the problem is the least k for which a solution of the required kind exists.

Quiescent state L. The initial state of every cell except cell 1: a soldier waiting for orders. By the quiescence conditions below, a quiescent cell stays quiescent as long as its neighbours are quiescent (or its right neighbour is the border).

General G. The initial state of cell 1, which starts the synchronization. After time 0 the rules of this paper also use G as an ordinary working state.

Firing state F. The target state, which all cells must enter at the same time.

Working states W . $W := S \setminus \{F\}$, the states used before firing.

Auxiliary states. The working states other than L and G. For $k = 5$ they are A and B, so that $S = \{L, G, A, B, F\}$ and $W = \{L, G, A, B\}$; for $k = 4$ the only auxiliary state is A; for general k they are $A_1, \dots, A_{k-3}$ (Section 4).

Border $*$. A symbol not in S : cell 1 sees it as its left neighbour and cell n as its right neighbour.

Neighbourhood. The triple $e = (x, y, z)$ of the states of the left neighbour, of the cell itself and of the right neighbour; x and z may be $*$. In files and figures a neighbourhood is also written as a word, and “ $xyz > d$ ” stands for the transition $\delta(x, y, z) = d$.

Rule δ ; transition; rule table. A *rule* (local transition function) is a map $\delta : (W \cup \{*\}) \times W \times (W \cup \{*\}) \rightarrow S$ satisfying the quiescence conditions; it gives the next state of a cell as a function of its neighbourhood (Section 2). A *transition* is one value $\delta(e) = d$, and the *rule table* lists all of them (Tables C.8 to C.10). Transitions from F or with F in the neighbourhood are irrelevant and left unspecified; the neighbourhoods $(*, y, *)$ occur only in the line of length 1 and are not counted. The *free transitions* are the remaining ones minus the

two fixed by the quiescence conditions: $5 \cdot 4 \cdot 5 - 4 - 2 = 94$ for five states and $4 \cdot 3 \cdot 4 - 3 - 2 = 43$ for four states. A *partial rule* is defined on some of the neighbourhoods only, typically on those it actually uses.

Quiescence conditions. $\delta(L, L, L) = \delta(L, L, *) = L$, see (1). The *left quiescence condition* $\delta(*, L, L) = L$, often assumed in the literature, is *not* assumed in this paper.

Line of length n . The cells $1, \dots, n$ with the border on both sides, started at time 0 in the *initial configuration* GL^{n-1} .

Configuration. The word of the states of all cells at one time, read from left to right.

Space-time diagram. The map $(t, i) \mapsto$ state of cell i at time t : C_n for the line of length n , C_∞ for the half-line (Section 2). A pair (t, i) is also called a *cell* of the diagram. In all figures time runs downwards, so that row t is the configuration at time t .

Synchronize; fire; firing time; fail. A cell *fires* when it enters F. A rule *synchronizes the line of length n at time T* if all n cells are in F at time T and no cell is in F earlier; T is the *firing time* and the configuration F^n at time T the *firing row*. A rule *fails* at length n if it does not synchronize the line of length n at time $2n - 2$ (in our examples, because some cell fires too early).

Firing squad synchronization problem (FSSP). Find a rule that synchronizes every line of length $n \geq 2$; such a rule is a *solution*. (The line of length 1 only needs $\delta(*, G, *) = F$.)

Minimal time; minimal-time solution. No solution synchronizes the line of length $n \geq 2$ before time $2n - 2$ (Remark 2.4), which is therefore the *minimal* (optimal) time. A *minimal-time solution* synchronizes the line of length n at time exactly $2n - 2$ for every $n \geq 2$; a *non-minimal-time* solution needs more time for some n .

$\mathcal{N}$ -minimal. For a set $\mathcal{N} \subseteq \{2, 3, \dots\}$ of lengths, a rule is *$\mathcal{N}$ -minimal* if it synchronizes the line of length n at time $2n - 2$ for every $n \in \mathcal{N}$; for instance $\{2, \dots, N\}$ -minimal; this property is *$\mathcal{N}$ -minimality*. The *bounded problem* asks for an $\mathcal{N}$ -minimal rule with $\mathcal{N}$ finite.

Restricted families of lengths. A solution for a family of lengths (for instance the powers of two) synchronizes only the lines whose lengths belong to the family, possibly not in minimal time.

Ring; symmetric rule. In the *ring* variant of the problem the cells form a cycle and there is no border. A rule is *symmetric* if $\delta(x, y, z) = \delta(z, y, x)$ for all neighbourhoods.

Mirror image. The mirror image of a rule δ is $\bar{\delta}(x_1, x_2, x_3) := \delta(x_3, x_2, x_1)$; it acts on the line reflected by $i \mapsto \bar{i} := n + 1 - i$ (Remark 3.7).

Renaming; labelling. A *renaming* is a bijection π of the states; applied to a rule or a diagram it replaces every state x by $\pi(x)$. Two diagrams coincide up to renaming if one is the image of the other; the images of one diagram under different renamings are its *labellings*.

Pre-firing configuration. The configuration at time $2n - 3$, one step before firing;

it is *uniform* if all cells are in the same state (for instance A^n).

Elementary cellular automaton; rule 22. An elementary CA has the two states 0, 1 and the neighbourhoods of three cells; a rule φ is identified by its *Wolfram number* $\sum_{x,y,z \in \{0,1\}} \varphi(x,y,z) 2^{4x+2y+z}$ [28]. Rule 22 sets a cell to 1 if and only if exactly one of the three cells of its neighbourhood is 1.

Local CA simulation. A formal notion of Nguyen and Maignan [12] under which one cellular automaton simulates another; they use it to generate families of six-state solutions.

Mazoyer’s solution. The six-state minimal-time solution of [8]. We use the rule table extracted mechanically from Duprat’s formalization of its correctness proof in the proof assistant Coq [16].

The rules δ_{12} , δ_{13} , δ_{14} , $\delta_{14}^b, \dots, \delta_{14}^e$. Explicit five-state rules found in this work (Section 6 and Appendix B.1). The index is the largest N for which the rule is $\{2, \dots, N\}$ -minimal (12, 13, and 14 for the last five); each of them fails at length $N + 1$.

Lucky rule. A rule returned by a solver for a finite set $\mathcal{N}$ of lengths that is $\mathcal{N}$ -minimal but fails at some length outside $\mathcal{N}$ (Section 6).

Frontier. The largest N for which a five-state $\{2, \dots, N\}$ -minimal rule is known; at present $N = 14$.

Picture vocabulary. In Figures 1 to 4 and 7 a cell is drawn as a *soldier*, a line is a *squad*, a state a *mood*, a time step a *tick*, the rule the *rule book* and the border a *wall*; the *hare* and the *tortoise* are signals of speeds 1 and $1/3$, and the *echo* is the reflected wave (Appendix A.3).

Appendix A.3. Geometry of space-time diagrams

Anti-diagonal; diagonal. The *anti-diagonal* $t + i = M$ (“anti-diagonal M ”) is the set of cells (t, i) with $t + i = M$; “below the anti-diagonal M ” means $t + i \leq M$. A *diagonal* is a set of cells with $t - i$ constant.

Predecessors; locality. The *predecessors* of a cell (t, i) with $t \geq 1$ are $(t - 1, i - 1)$, $(t - 1, i)$ and $(t - 1, i + 1)$; its state is δ of their states. Hence (*locality*) the state of a cell at time $t + P$ depends only on the cells within distance P of it at time t .

Backward dependency cone. The set of cells reached from a given cell by repeatedly passing to predecessors; only these can influence its state.

Light cone. The cells (t, i) with $t \geq i - 1$. Every cell outside it is quiescent (Lemma 2.1): information travels at most one cell per step.

Signal; speed. A *signal* is a recognisable pattern that moves through a diagram along a straight line. Its *speed* is the number of cells it moves per step: speed 1 is the maximal speed, speed $1/3$ means one cell every three steps, and a *rational speed* is a speed in $\mathbb{Q}$. A signal is *emitted* where it starts and *reflected* when it bounces off a border.

Front. The cells $(i - 1, i)$, $i \geq 1$, on the edge of the light cone; $f_i := C_\infty(i - 1, i)$ is the state of the front at cell i (Lemma 2.3). In a minimal-time solution every cell i leaves $\mathbb{L}$ exactly at time $i - 1$: the front moves at speed one.

Wake. The cells immediately behind the front.

Region; background; boundary. A *region* is a set of cells of a diagram, usually delimited by signals, its *boundaries*; the *background* is the pattern that fills a region, for instance the quiescent background ahead of the front.

Doubly periodic; periodic region; checkered region. A region is *doubly periodic*, or *periodic in time and space*, with periods $p, q \geq 1$ if $C(t+p, i) = C(t, i)$ and $C(t, i+q) = C(t, i)$ whenever the cells involved lie in it; a *periodic region* (or zone) is a doubly periodic one, and a *checkered* region is one in which two states alternate in both directions, like $ALAL \dots$.

Wedge. A set $\Omega = \{(t, i) : s \leq i \leq \alpha t, t \geq t_0\}$ with $s \geq 1$ and $\alpha > 0$ (Theorem 3.6): an unbounded angular region at the left border.

Boundary layer. The cells between the left border and a periodic region that starts at a bounded distance from the border.

Half-line; C_∞ . The cells $1, 2, 3, \dots$ with the border on the left only, started in $GLL \dots$; C_∞ is its space-time diagram (Section 2). The *half-line conditions* are the constraints on C_∞ alone used in this paper: no cell of C_∞ fires, see (2), the front is never L (Lemma 2.3), and $PUMP(n, n')$ holds (Appendix A.4).

Agreement. $C_n(t, i) = C_\infty(t, i)$ whenever $t + i \leq 2n - 2$ (Lemma 2.2): below this anti-diagonal the line of length n cannot distinguish itself from the half-line.

Reflected triangle R_n . $R_n = \{(t, i) : 1 \leq i \leq n, t + i \geq 2n - 1, t \leq 2n - 2\}$, the part of C_n that is influenced by the right border: the triangle between the anti-diagonal $2n - 1$, the right border and the firing row.

Return chain; reflected wave. The *return chain* consists of the cells $(2n - 1 - i, i)$, $1 \leq i \leq n$, of the anti-diagonal $2n - 1$; by Lemma 2.3, C_n and C_∞ differ at each of them. The *reflected wave* is the signal of speed one that runs along it, from the right border at time $n - 1$ to the left border at time $2n - 2$.

Relative coordinates (τ, κ) . For the line of length n , $\tau := t - (n - 1)$ is the time elapsed since the front reached the right end and $\kappa := n - i$ is the distance to the right end; then $R_n = \{(\tau, \kappa) : 0 \leq \kappa \leq \tau \leq n - 1\}$ (Section 3).

Input anti-diagonals; input cells; input word J_n . The anti-diagonals $t + i = 2n - 3$ (*lower*) and $t + i = 2n - 2$ (*upper*), on which $C_n = C_\infty$ and from which R_n is computed. Their cells are the *input cells*, and $J_n(\kappa)$, $0 \leq \kappa \leq n - 1$, is the pair (lower, upper) of input cells at distance κ from the right end.

Determinacy; Ψ . R_n is obtained from J_n and the two borders by one partial map Ψ that depends on δ but not on n (Lemma 3.1).

Cone $\text{cone}_n(\tau, \kappa)$; right-anchored. The set of input cells in the backward dependency cone of the relative cell (τ, κ) of R_n ; it is *right-anchored* if the backward dependency cone does not reach the left border (Lemma 3.2).

Co-moving frame; depth; $\Phi_t(j)$; depth-row. The half-line seen from the front: $\Phi_t(j) := C_\infty(t, t + 1 - j)$ is the cell at *depth* j behind the front at time t ($0 \leq j \leq t$). The *depth-row* j is the sequence $t \mapsto \Phi_t(j)$, which is eventually periodic; $\theta(j)$ denotes its transient. The *non-periodic part* of the half-line consists of the cells $(t, t + 1 - j)$ with $t < \theta(j)$, i.e. of the transient parts of all depth-rows. In this frame information travels only towards larger depths (a *one-sided* automaton, (3)).

Chains $v_c(i)$. For the line of length n and $c \geq -1$, the *chain* c is the anti-diagonal $t+i = 2n-2+c$, and $v_c(i) := C_n(2n-2+c-i, i)$ is its cell at position i . Chains -1 and 0 are the input anti-diagonals, chain 1 is the return chain, and cell i fires on chain i (Section 3).

Skewed coordinates $u_c(y)$; slice. $u_c(y) := v_c(y + \lfloor c/2 \rfloor)$, with $\varepsilon_c = 1$ for even c and $\varepsilon_c = 0$ for odd c in the recurrence (5). The *slice* y is the sequence $(u_1(y), u_2(y), \dots)$, and $\mathbf{u}(y) := (u_1(y), \dots, u_{2s}(y))$ is its initial segment of length $2s$ (proof of Lemma 3.5).

Finite-state transducer; cascade. A (sequential) finite-state transducer is a finite automaton that reads a sequence of symbols and writes one symbol for each symbol read; a *cascade* is a sequence of transducers in which each one reads the output of the previous one.

Division points; hierarchy of signals. In the classical constructions the line is cut at *division points* near $n/2, n/4, \dots$, marked by the crossings of the reflected wave with a hierarchy of slower and slower signals, of speeds $1/(2^j - 1)$, emitted by the general; the pieces are then synchronized recursively.

Descriptive words. *Self-similar* or *hierarchical*: made of nested, recursively repeated patterns, like the diagrams of Mazoyer’s solution. *Chaotic*: without visible periodic, eventually regular or self-similar organisation, like rule 22 (no formal notion, such as positive entropy, is claimed). *Architecture* of a half-line: the qualitative arrangement of its regions and signals. *Firing wave*: the pattern that runs ahead of the firing row, for instance the G’s in the pre-firing configurations $\mathbf{L} G^{n-2} \mathbf{B}$ of δ_{14} . *Marker*: a state or pattern that singles out particular cells.

Appendix A.4. The two barriers

Pumping lemma; PUMP(n, n'). For a minimal-time solution and $4 \leq n < n'$, the input words J_n and $J_{n'}$ differ somewhere on $\text{cone}_n(n-1, 0)$ (Lemma 3.3); PUMP(n, n') denotes this condition, which only involves the half-line. “Pumping constraints up to N' ” means PUMP(n, n') for all $4 \leq n < n' \leq N'$.

Speed-1/3 barrier. Theorem 3.4: in a minimal-time solution the transients of the depth-rows satisfy $\limsup_{j \rightarrow \infty} \theta(j)/j \geq 3/2$, i.e. along infinitely many depth-rows the transient part of the half-line reaches, up to an arbitrarily small angle, the line $i = t/3$.

Left-border lemma. Lemma 3.5: if a k -state rule synchronizes the line of length n at time $2n-2$, and the upper and lower input anti-diagonals, read by position, are L -periodic on $[s, Y]$ and on $[s, Y-1]$ respectively (with $s \geq 1$ and $s+L \leq Y \leq n-s$), then $Y-s \leq (k-1)^{2s}L$.

Left-border barrier. Theorem 3.6: the half-line of a minimal-time solution is not doubly periodic on any wedge.

Eventually regular; junction words; blocks. (Definition before Corollary 3.8.)

A half-line is *eventually regular* if there are a time t_1 and a period $P \geq 1$ such that, for each residue $r \in \{0, \dots, P-1\}$ and every $\nu \geq 0$, the configuration at time $t_1 + r + \nu P$ is $w_0 z_1^{a_1 + \nu D_1} w_1 \dots z_m^{a_m + \nu D_m} w_m$ with words w_j and nonempty words z_j over W and integers $a_j, D_j \geq 0$ that depend on r only. The w_j are the

junction words (they carry the signals), the $z_j^{a_j+\nu D_j}$ are the *blocks* (the periodic regions), $|z_j|$ is the *block period* and D_j the *growth* of block j per period. In Section 6.3 a half-line for which the certificate below fails is called *irregular*; this only records the failure of the certificate, not a proof that the half-line is not eventually regular.

Regularity certificate. The finite test of Lemma 3.9, which proves that a given half-line is eventually regular (program `germpersist.py`).

Appendix A.5. Germs

Germ; complexity $c_M(\delta)$. The *germ* of a rule δ below M is the restriction of δ to the neighbourhoods that occur at the cells (t, i) of C_∞ with $t \geq 1$ and $t + i \leq M$; its *complexity* $c_M(\delta)$ is the number of these neighbourhoods other than (L, L, L) (Section 6.3). A germ can also be given on its own, as a list of transitions.

Determined; extended half-line; closed, open. A germ *determines* the half-line up to an anti-diagonal beyond M if the simulation of C_∞ with the transitions of the germ only never meets a neighbourhood outside the germ on the cells below that anti-diagonal; the diagram so obtained is the *extended half-line*. Program outputs call the germ **CLOSED** in this case and **OPEN** otherwise.

Anti-diagonal order. The order of the cells of C_∞ by anti-diagonal and, within an anti-diagonal, by increasing time. The enumeration of germs follows it, and its symmetry breaking requires that **A** occurs before **B** in this order.

Completion; completion test. A *completion* of a germ is a rule that contains it. The *completion test for the lengths $2, \dots, N$* asks a SAT solver whether some completion is $\{2, \dots, N\}$ -minimal and satisfies the half-line conditions.

Refuted; survivor; filter. A germ is *refuted* once it is proved that none of its completions is a minimal-time solution. The *survivors* of a sequence of tests (*filters*) are the germs that none of the tests refutes.

Appendix A.6. Propositional logic, solvers and certificates

Variable; literal; clause; CNF. A propositional variable takes the value true or false; a *literal* is a variable X or its negation $\neg X$; a *clause* is a disjunction ($\vee$) of literals, and a *unit clause* has a single literal. A formula in *conjunctive normal form* (CNF) is a conjunction of clauses; an *instance* is a formula given to a solver.

Model; SAT; UNSAT; UNKNOWN. A *model* is an assignment of truth values that makes the formula true. A formula is *satisfiable* (SAT) if it has a model and *unsatisfiable* (UNSAT) otherwise. UNKNOWN reports that a solver reached its time limit without an answer; the instance is then *undecided*. A model is *decoded* into a rule table by reading off the true variables $T_{e,d}$.

One-hot encoding. A group of variables of which exactly one is true; it encodes a quantity with finitely many values.

The formula $MT_k(\mathcal{N}; M, N')$. The formula of Section 4, which is satisfiable whenever a k -state minimal-time solution exists. Its variables are $T_{e,d}$ (“ $\delta(e) = d$ ”) and $X_{\xi,w}$ (“the cell ξ is in state w ”); it contains the *transition clauses* for the reflected triangles of the lengths in $\mathcal{N}$ and for the half-line up to the anti-diagonal

M , and the pumping constraints up to N' (encoded with auxiliary *difference variables*). Optional parts are the return-chain constraints of Lemma 2.3, the *determinacy links* (“links”) with variables $E_{n,n'}(\kappa_0)$, “ J_n and $J_{n'}$ coincide on the positions $0, \dots, \kappa_0$ ”, and symmetry breaking. The *time-major order* of cells is by time, then position. A *soundness check* verifies that a known solution (Mazoyer’s) satisfies the formula, which tests that no constraint excludes a genuine solution.

Symmetry breaking. Constraints that remove all but one member of each class of equivalent solutions, here the renamings of the auxiliary states.

Unit propagation. The inference that repeatedly sets the literal of any clause whose other literals are all false.

SAT solver; CDCL; conflict. A SAT solver decides whether a CNF formula is satisfiable. Kissat [23] and CaDiCaL [24] implement *conflict-driven clause learning* (CDCL): they assign variables, propagate, and at every *conflict* (a clause whose literals are all false) learn a new clause and backtrack. The number of conflicts measures the effort, and a *conflict budget* limits it. PySAT [25] is a Python interface to such solvers.

Incremental solving; assumptions. Solving a sequence of related formulas with one solver that keeps what it has learned; *assumptions* are literals fixed for a single call.

Selector variables. One auxiliary variable x_g per germ g , with the clauses $\neg x_g \vee \ell$ for every literal ℓ that fixes a transition of g , and the clause $\bigvee_g x_g$. A model with x_g true satisfies the transitions of g ; hence the extended formula is unsatisfiable if and only if, for every germ g , the original formula together with the transitions of g is unsatisfiable, and one certificate covers the whole list (Proposition 6.4).

Unsatisfiable core. A subset of the clauses, or of the assumptions, that is already unsatisfiable.

Proof certificate; DRAT; LRAT; certified. A proof of unsatisfiability given as a list of clauses, each of which follows from the formula and the preceding clauses by a mechanically checkable rule (reverse unit propagation, or the resolution-asymmetric-tautology property), ending with the empty clause. The DRAT format also allows clause deletions. The LRAT format [26] also records, for each clause, the earlier clauses from which it follows, so that it can be checked without any search; `lratt-check`, of the `drat-trim` suite [27], is such a checker. A claim of unsatisfiability is *certified* when its LRAT proof has been checked (output “VERIFIED”). A *computer-assisted proof* is a proof some of whose steps are finite computations carried out by programs; in this paper every unsatisfiability claim used in such a proof is certified.

SHA-256 digest. A cryptographic hash of a file, listed in Appendix D so that readers can check that they use the same files.

Counterexample-guided search over lengths. Solve for a set of lengths, simulate the model, and if it fails at a new length, add that length and repeat (Appendix B.1).

Logic-based Benders decomposition; nogood. A scheme in which a master problem proposes partial solutions (here germs), a subproblem (here the comple-

tion test) refutes them, and a *nogood*, a clause that excludes the refuted proposal or a generalisation of it, is added to the master problem.

Depth-first search; nodes; exhaustive; pipeline. A depth-first search extends a partial solution one decision at a time and backtracks when a test fails; its *nodes* are the partial solutions visited. A *breadth-first* search visits them level by level instead. A search or enumeration is *exhaustive* if it covers all candidates. The *pipeline* of Section 6.3 is the sequence of filters applied to a list of germs.

Look-ahead search tree; random-probe estimator. The tree of partial rule tables in which the half-line transitions are fixed in the order of their first use, a node being discarded when the solver refutes it within a conflict budget. Knuth’s estimator [29] follows random root-to-leaf paths (*probes*); if $d_1, \dots, d_D$ are the numbers of children of the nodes of a probe before its leaf, then $1 + d_1 + d_1 d_2 + \dots + d_1 \dots d_D$ is an unbiased estimate of the number of nodes, and the estimates are averaged over the probes.

Cube-and-conquer. Splitting a formula into many subproblems, each given by a partial assignment (a *cube*), that are solved independently [30]; the cubes are produced by the look-ahead solver `march_cu`, and a cube is *refuted* when the formula together with it is unsatisfiable. Together the cubes cover all assignments, so the formula is unsatisfiable if and only if every cube is refuted.

Brute-force checker. The program `fsspcheck.c`, which simulates each line directly from the rule table and uses none of the lemmas.

Wall-clock time; CPU-hours. The elapsed real time of a run; one CPU-hour is one hour of one processor core.

Table A.6: Symbols, in order of first use.

symbol	meaning	where
$S, k = S $	set of states, number of states	Section 2
L, G, F	quiescent state, general, firing state	Section 2
W	working states $S \setminus \{F\}$	Section 2
$A, B; A_1, \dots, A_{k-3}$	auxiliary states ($k = 5$; general k)	Sections 2 and 4
$*$	border symbol	Section 2
$\delta, \bar{\delta}$	rule; its mirror image	Section 2, Remark 3.7
$e = (x, y, z), d$	neighbourhood; value $\delta(e) = d$	Section 4
n, n'	lengths of lines	Section 2
t, i, T	time, position, a particular time	Section 2
C_n, C_∞	diagrams of the line of length n and of the half-line	Section 2
$\mathcal{N}, N$	set of lengths; largest length	Section 2
f_i	front, $C_\infty(i - 1, i)$	Lemma 2.3
R_n	reflected triangle	Section 2

symbol	meaning	where
τ, κ	relative time and distance to the right end	Section 3
$J_n(\kappa)$	input word of R_n	Section 3
Ψ	partial map computing R_n from J_n	Lemma 3.1
$\text{cone}_n(\tau, \kappa)$	input cells in the backward cone of (τ, κ)	Section 3
$\text{PUMP}(n, n')$	the pumping condition for the lengths $n < n'$	Section 3
$\Phi_t(j), j$	co-moving frame; depth	Section 3
$\theta(j), P_j$	transient and a period of the depth-row j	Theorem 3.4
$v_c(i), c$	cell i of chain c ; chain index	Section 3
$u_c(y), \varepsilon_c, y$	skewed coordinates; slice index	Section 3
s, L, Y	start, period and end of a periodic stretch	Lemma 3.5
$\mathbf{u}(y), \Gamma, \Gamma', y_1, y_2,$ h	local to the proof of Lemma 3.5	Lemma 3.5
$\Omega, \alpha, t_0, p, q, \gamma$	wedge, its slope and start; periods; a constant	Theorem 3.6
t_1, P, r, ν	start, period, residue, period count	Corollary 3.8
w_j, z_j, a_j, D_j, m	junction and block words, exponents, growth, number of blocks	Corollary 3.8
$j(r), s_r, \beta$	local to the proof of Corollary 3.8	Corollary 3.8
$X(\mathbf{e}), \mathbf{1}_j, \Lambda, q_j, \ell_\nu$	block word, unit vector, P steps of the half-line; local to Lemma 3.9	Lemma 3.9
$\bar{i}$	mirrored position $n + 1 - i$	Remark 3.7
$\text{MT}_k(\mathcal{N}; M, N')$	the formula; M : half-line bound, N' : pumping bound (omitted: no pumping)	Section 4
$T_{e,d}, X_{\xi,w}, \xi$	transition and cell variables; a cell	Section 4
$E_{n,n'}(\kappa_0)$	determinacy link variable	Section 4
$c_M(\delta)$	complexity of the germ of δ below M	Section 6.3
$\delta_{12}, \dots, \delta_{14}^e$	the five-state rules of Section 6 and Appendix B.1	Section 6, Appendix B.1

Appendix B. Further experiments

This appendix records the experiments summarised in Section 6. Every rule was checked with the brute-force checker `fsspcheck.c` and with an independent simulation. Unsatisfiability claims called *certified* come with LRAT proofs checked by `lrat-check`; all other solver results are reported as the solvers returned them.

Appendix B.1. Four more rules for all lengths up to fourteen

A second rule for all lengths up to fourteen, δ_{14}^b (file `delta14b.txt`), was found in 29 s on another germ of complexity 16. Its half-line is of the periodic

kind: a boundary layer of four cells followed by a region that is periodic in time and space. With a boundary layer of four cells the bound of Lemma 3.5 (with $s = 5$) lies far beyond the lengths we can simulate, but the half-line is eventually regular: the certificate of Lemma 3.9 succeeds with $t_1 = 600$ and $P = 12$ (program `germpersist.py`). Hence no completion of this germ is a minimal-time solution.

Two of the 16 germs that survive the classification of Section 6.3 have complexity exactly 15, and their half-lines coincide on all cells with $t \geq 1$ (checked for $t \leq 1500$) after the renaming $G \mapsto A$, $A \mapsto B$ of the states of the first one: the first reuses the state G of the general on its half-line and never uses B there, the second never uses G after time 0. A counterexample-guided search completes both of them to rules for all lengths up to 14: δ_{14}^c (file `delta14c.txt`) and δ_{14}^d (file `delta14d.txt`, found in 376 s). Raising the bound of Proposition 6.4 to 16 therefore amounts to refuting this one half-line in its two labellings, and a completion test can only do so with lines of length at least 15 (Appendix B.4).

Among the germs of complexity 17 (Appendix B.3), the first one that passes the completion test for the lengths $2, \dots, 13$ extends to a fifth rule for all lengths up to 14, δ_{14}^e (file `delta14e.txt`, found in 401 s); its half-line is irregular.

Table B.7 compares the five rules. All of them end with a configuration before firing that is uniform, or uniform except at the end cells. All five fail at $n = 15$ in the same way: the first cells to fire are in the right half of the line, too early by one to seven steps. For δ_{14} the mechanism behind the configurations $L G^{n-2} B$ provably does not extend: with its germ fixed, no rule synchronizes all lengths $2, \dots, 15$ with these configurations before firing (Kissat, unsatisfiable after about 30 minutes). The corresponding tests for the uniform configurations of δ_{14}^c and δ_{14}^d were stopped without an answer after about 2.5 hours each.

rule	c_{78}	half-line	before firing	first firing at $n = 15$	
			($4 \leq n \leq 14$)	time	cells
δ_{14}	16	chaotic (rule 22)	$L G^{n-2} B$	22	13–15
δ_{14}^b	16	eventually regular	$G^{n-1} A$	25	12
δ_{14}^c	15	irregular	G^n	21	11
δ_{14}^d	15	that of δ_{14}^c , renamed	A^n	22	11–15
δ_{14}^e	17	irregular	A^n	27	9–11

Table B.7: The five rules that synchronize every line of length at most 14 in minimal time. The correct firing time for $n = 15$ is $t = 28$.

Appendix B.2. Germs of complexity at most sixteen

The pipeline of Proposition 6.4, run on the 9,787 germs of complexity at most 16 ($1.5 \cdot 10^9$ nodes), gives the numbers of Table 5. The completion test for the lengths $2, \dots, 10$ refutes 8,824 germs (one LRAT proof of 2.0 GB, found in 556 s and checked in 51 s). Of the 963 survivors, 931 are refuted by Lemma 3.5 or Lemma 3.3 applied to their half-lines, each of which is determined by the germ far enough for the lemma to apply (we simulate up to anti-diagonal 10^4 , and for two germs the input anti-diagonals of the line of length 20,000). Used alone, the half-line arguments refute at least 8,263 of the 9,787 germs, without any SAT call. Among the 32 germs that pass both filters (one of which is not determined beyond anti-diagonal 94) are germs with chaotic or self-similar half-lines, on which Lemma 3.5 has no grip, and germs whose periodic regions start too far from the border for the bound of the lemma to be reached within our simulation range. The completion test for the lengths $2, \dots, 13$ refutes 8 of them (one LRAT proof of 1.4 GB, found in 318 s and checked in 33 s); 23 of the remaining 24 do have completions for these lengths (the last one is undecided after 900 s). Of these 24, 8 have eventually regular half-lines (certified with periods $P \leq 48$) and are excluded by Lemma 3.9, although 7 of them admit completions for all lengths up to 13. For the last 16 germs, among them the germ of δ_{14} , the certificate fails (periods $P \leq 48$, block periods at most 24, starting times up to 1500); the half-lines we inspected are chaotic, and none of our half-line arguments applies to them. Their completion problems become hard at length 14: for most of them the lengths $2, \dots, 14$ are undecided after 40 minutes.

Appendix B.3. Germs of complexity seventeen

The enumeration with bound 17 visits $6.1 \cdot 10^9$ nodes and returns 53,081 germs; those of complexity at most 16 are exactly the 9,787 germs of Table 5, and 43,294 have complexity 17. Here we applied the filters in a different order. On the half-lines extended to anti-diagonal 1300, Lemma 3.5 refutes 34,613 of them and Lemma 3.3 (lengths up to 650) another 637. The completion test for the lengths $2, \dots, 10$ refutes 6,834 of the remaining 8,044 (one LRAT proof of 1.9 GB, found in 440 s and checked in 33 s). Extending the half-lines of the 1,210 survivors to anti-diagonals 3300 and 10^4 , the two lemmas refute 903 more, and the regularity certificate of Lemma 3.9 excludes 120 of the rest (periods $P \leq 48$, block periods at most 24). This leaves 187 germs of complexity 17. We ran the completion test for the lengths $2, \dots, 13$ on 30 of them only: 11 are refuted, 6 are undecided within 10^6 conflicts, and 13 have completions, one of which yields δ_{14}^e .

Appendix B.4. Lines of length fifteen for the complexity-15 half-line

We split the completion problems of the two labellings of the complexity-15 half-line (Appendix B.1) for the lengths $2, \dots, 15$ into 2,731 (first labelling) and 3,070 (second labelling) cubes with the look-ahead solver `march_cu` (depth 12) [30] and solved the cubes with CaDiCaL under assumptions and, for the hard ones, with Kissat on the formula extended by the cube, which was much faster on them. After about sixteen CPU-hours no cube is satisfiable. Every cube of the first labelling has been attempted: 2,694 are refuted and 37 resisted the limits we set (up to $2 \cdot 10^6$ conflicts, or five to ten minutes of Kissat). Of the cubes of the second labelling, 1,376 are refuted, 375 resisted these limits and 1,319 were not attempted. Both problems therefore remain undecided. Single runs on the whole problems were not more successful: Kissat on the second labelling with the lengths $2, \dots, 15$ stopped without an answer after two hours, and on the first labelling with the lengths $2, \dots, 14$ and the non-firing constraints of the partial triangles of the lengths $15, \dots, 26$ up to relative time 12 after three hours.

Appendix B.5. Unsatisfiable cores

For the periodic germs, the completion problem is decided in less than a second, but its unsatisfiable cores are large: for the germ of the rule of Table 4 with lengths $2, \dots, 10$, completion required for the lengths $2, \dots, 12$, the core returned by CaDiCaL contains 38 of its 39 transitions, and shrinking it requires solving exactly the hard subproblems (each deletion step of a minimization takes about a minute). Nogoods learned from the completion problem, as in a logic-based Benders decomposition, would therefore be almost as specific as the germs themselves.

Appendix C. Rule tables

The three tables give the rules δ_{12} , δ_{13} (Theorem 6.1) and δ_{14} (Proposition 6.3) in full; the other rules are in the files named in Appendix B.

Appendix D. Reproducibility

The directory `fssp5/` of the accompanying repository contains all programs, the rule tables and the logs. The main components are:

`src/fsspcheck.c`

independent brute-force simulator: reads a rule table and checks minimal-time synchronization for a range of lengths (full simulation of every line, no use of the agreement lemma);

`src/gencnf.py`

CNF generator for $\text{MT}_k(\mathcal{N}; M, N')$ with the options of Section 4 (return chains, pumping, links, symmetry breaking) and the restricted classes used in the experiments;

`src/pumping.py`

computation of the cones of Lemma 3.2 by breadth-first search and direct verification of $\text{PUMP}(n, n')$ on a given rule;

`src/fsspdfs.c`

exhaustive depth-first enumeration of partial rules (Table 3);

`src/mazoyer_from_coq.py`

extraction of Mazoyer's rule from the file `autom.v` of Duprat's Coq development [16];

`src/validate_mazoyer.py`

soundness test of the encoding;

`src/check_halfline.py`

direct check that a half-line diagram has no firing cell below a given anti-diagonal;

`src/fixgerm.py`, `src/knuth.py`, `src/benders.py`

the experiments of Section 6;

`src/germdfs.c`

enumeration of the half-line germs of bounded complexity (Proposition 6.4);

`src/germinc.py`, `src/germcert.py`

completion tests of germs (CaDiCaL under assumptions) and a single LRAT certificate for a list of refuted germs;

`src/germext.c`, `src/germext2.c`

extension of the half-line determined by a germ, and the checks of Lemma 3.5 and Lemma 3.3 on it (`germext2.c`: streaming version of the check of Lemma 3.5 for a single long line);

`src/germcompare.py`

the neighbourhoods used by the half-line of a rule below a given anti-diagonal (Theorem 6.1(c)).

The directory `results/germs` contains the germ lists, the logs of the classification and, in `CERTIFICATES.txt`, the digests of the certificates.

Certificate for Theorem 5.1.

```
python3 gencnf.py 4 9 mt4_9.cnf
cadical --lrat --binary=false mt4_9.cnf mt4_9.lrat # UNSAT
lrat-check mt4_9.cnf mt4_9.lrat # VERIFIED
```

SHA-256 digests (each on two lines):

```
mt4_9.cnf          d18d65480e77505193b8f60362badad5
                   e71f8f08505f29befbcd3b2b18cdaabf
mt4_9.lrat         1d511a75ab25587fa4a3476b95ad3f71
                   33c7375ad96661fd22fa598aec6c8092
uniformA_2-12.txt  b62abbcac0653a661cd1bbe344dee9db
                   ba0cddee37448ea16681d436166c3aad
delta13.txt        99dbe526792323b2f77786bcb3dee4fb
                   ee624e2efac5dc8a52c02cb8fb45b891
mazoyer6.txt       4b0717ea7319957c1fdcc887fe09c107
                   4cf338e8b849cdd32b4307d0083f417e
```

Checking Theorem 6.1.

```
R=results/uniformA_2-12.txt
fsspcheck $R 2 12 # OK for all 2 <= n <= 12
fsspcheck $R 13 13 # FAIL: a cell fires at time 23
python3 pumping.py $R 40 # 0 violations
python3 check_halflines.py $R 78 # no firing cell, t+i <= 78
S=results/delta13.txt
fsspcheck $S 2 13 # OK for all 2 <= n <= 13
fsspcheck $S 14 14 # FAIL: a cell fires at time 21
python3 pumping.py $S 40 # 0 violations
python3 check_halflines.py $S 78 # no firing cell, t+i <= 78
python3 germcompare.py $R $S 78 # identical, 22 besides (L,L,L)
```

The uniform configurations before firing in (a) and (b) are visible with the option `-v` of the checker, which prints the diagrams.

Checking Corollary 6.2 and Propositions 6.3 and 6.4.

```
gcc -O2 -o germext germext.c && gcc -O2 -o germdfs germdfs.c
# Corollary 6.2: germ of delta12/delta13 (22 neighbourhoods)
./germext 5 1300 3 6 < ../results/germs/germ_delta12.txt
# CLOSED; REFUTED n=518 s=2 L=1 Y=259
# Proposition 6.3
./fsspcheck ../results/delta14.txt 2 14 # OK
./fsspcheck ../results/delta14.txt 15 15 # FAIL: fires at time 22
./germext 5 10000 3 24 800 < ../results/germs/germ_delta14.txt
# CLOSED, no refutation
# Proposition 6.4 (complexity <= 14)
./germdfs 5 78 14 40 > germs_K14.txt # 244 germs
python3 germinc.py germs_K14.txt 10 78 # 230 UNSAT, 14 SAT
```

```
python3 germcert.py ../results/germs/K14_refuted10.txt 10 k14_n10
# UNSAT, VERIFIED
./germext 5 3300 3 12 < germs_K14_surv10.txt # 14 x REFUTED (s=1)
```

The files `results/germs/germ_delta12.txt` and `germ_delta14.txt` contain the two germs in the one-line format of `germdfs.c`, as printed by `germcompare.py -germline`; the complete pipeline for complexity at most 16 is `germpipe.sh`.

Search-size estimates (Section 6.4).

```
python3 knuth.py 9 16 30 3000 1 4 # 25.5 nodes (knuth_k4_N9.log)
python3 knuth.py 12 78 3 3000 # 6.7e18 (knuth_k5_N12.log)
python3 knuth.py 18 78 3 3000 # 2.2e18 (knuth_k5_N18.log)
```

The logs are in `results`, and the per-cube results of Appendix B.4 in `results/cubes`.

Validation against Mazoyer's solution.

```
python3 mazoyer_from_coq.py autom.v > mazoyer6.txt
fsspccheck mazoyer6.txt 3 3000 # OK for all 3 <= n <= 3000
python3 validate_mazoyer.py 14 # SAT under exactly one renaming
```

left	centre	right neighbour				
		*	L	G	A	B
*	L	—	A	A	A	A
L	L	L	L	L	B	A
G	L	A	G	G	L	A
A	L	B	A	B	A	A
B	L	G	B	L	A	A
*	G	—	A	A	A	A
L	G	B	B	G	L	B
G	G	G	G	G	A	B
A	G	A	L	A	A	G
B	G	B	G	L	A	A
*	A	—	G	B	F	L
L	A	A	L	A	A	B
G	A	B	L	A	B	A
A	A	F	B	G	F	A
B	A	A	B	A	G	A
*	B	—	A	A	A	A
L	B	G	B	L	B	B
G	B	A	A	B	B	G
A	B	L	B	A	A	G
B	B	G	A	B	B	L

Table C.8: The five-state rule δ_{12} of Theorem 6.1: entry $\delta(\text{left}, \text{centre}, \text{right})$. The entries $\delta(L, L, L) = \delta(L, L, *) = L$ are the quiescence conditions; “—” marks the unused neighbourhoods $(*, y, *)$. Machine-readable copy: **results/uniformA_2-12.txt**.

left	centre	right neighbour				
		*	L	G	A	B
*	L	–	L	G	L	G
L	L	L	L	L	A	G
G	L	B	G	L	B	A
A	L	A	A	B	A	A
B	L	A	A	L	B	B
*	G	–	A	G	A	G
L	G	B	B	G	B	L
G	G	A	G	G	B	B
A	G	A	L	B	A	A
B	G	F	G	L	G	L
*	A	–	G	B	F	F
L	A	A	L	G	L	G
G	A	A	L	A	A	A
A	A	F	G	A	F	L
B	A	G	A	B	A	L
*	B	–	A	G	G	G
L	B	G	B	L	G	G
G	B	L	B	A	A	A
A	B	F	B	F	A	B
B	B	A	B	A	L	G

Table C.9: The five-state rule δ_{13} of Theorem 6.1: entry $\delta(\text{left}, \text{centre}, \text{right})$. The entries $\delta(L, L, L) = \delta(L, L, *) = L$ are the quiescence conditions; “–” marks the unused neighbourhoods $(*, y, *)$. Machine-readable copy: **results/delta13.txt**.

left	centre	right neighbour				
		*	L	G	A	B
*	L	–	F	F	L	G
L	L	L	L	G	G	B
G	L	G	A	B	A	B
A	L	G	B	G	G	G
B	L	A	B	A	A	L
*	G	–	L	F	L	L
L	G	F	A	F	B	A
G	G	B	B	F	G	F
A	G	B	A	G	G	A
B	G	B	A	G	A	L
*	A	–	G	G	L	G
L	A	A	B	G	B	A
G	A	B	A	G	L	A
A	A	B	L	A	G	A
B	A	L	A	A	G	A
*	B	–	G	G	G	G
L	B	G	B	G	A	L
G	B	F	B	A	B	B
A	B	G	B	B	G	B
B	B	A	L	A	A	L

Table C.10: The five-state rule δ_{14} of Proposition 6.3: entry $\delta(\text{left}, \text{centre}, \text{right})$. The entries $\delta(L, L, L) = \delta(L, L, *) = L$ are the quiescence conditions; “–” marks the unused neighbourhoods $(*, y, *)$. Machine-readable copy: **results/delta14.txt**.